

MS - repetition - no_frac - Serial position analysis

```
# do this in calling R script
# library(ggplot2)
# library(fmsb) # for TestModels
# library(lme4)
# library(kableExtra)
# library(MASS) # for dropterm
# library(tidyverse)
# library(dominanceanalysis)
# library(cowplot) # for multiple plots in one figure
# library(formatters) # to wrap strings for plot titles
# library(pals) # for continuous color palettes

# load needed functions
# source(paste0(RootDir, "/src/function_library/sp_functions.R"))
# do this in the calling R script

# for testing
# CurPat <- "GM"
# CurTask <- "repetition"
# MinLength <- 4
# MaxLength <- 10
# BestModelIndexL1 <- 1
# BestModelIndexL2 <- 1
# BestModelIndexL3 <- 1
# ReportTitle <- paste(CurPat, " - ", CurTask, " - ", RunLabel, " - Serial position analysis")
# RandomSamples <- 10
# DoSimulations <- FALSE
# RemoveFracPres <- TRUE

CurPat <- params$CurPat
CurTask <- params$CurTask
MinLength <- params$MinLength
MaxLength <- params$MaxLength
BestModelIndexL1 <- params$BestModelIndexL1
BestModelIndexL2 <- params$BestModelIndexL2
BestModelIndexL3 <- params$BestModelIndexL3
ReportTitle <- params$ReportTitle
RandomSamples <- params$RandomSamples
DoSimulations <- params$DoSimulations
RemoveFracPres <- params$RemoveFracPres

# done in calling script
# print(paste0("Using root dir: ", RootDir))
# RunDir <- paste0(paste0(RootDir, "/output/"), RunLabel))
# if(!dir.exists(RunDir)){
#   dir.create(RunDir, showWarnings = TRUE)
#   print(paste0("created run directory: ", RunDir))
# }
```

```

# # create patient directory if it doesn't already exist
# PatientDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat)
# if(!dir.exists(PatientDir)){
#   dir.create(PatientDir, showWarnings = TRUE)
#   print(paste0("created participant directory: ", PatientDir))
# }
# TablesDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat, "/tables/")
# if(!dir.exists(TablesDir)){
#   dir.create(TablesDir, showWarnings = TRUE)
#   print(paste0("created tables directory: ", TablesDir))
# }
# FigDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat, "/fig/")
# if(!dir.exists(FigDir)){
#   dir.create(FigDir, showWarnings = TRUE)
#   print(paste0("created figures directory: ", FigDir))
# }
# ReportsDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat, "/reports/")
# if(!dir.exists(ReportsDir)){
#   dir.create(ReportsDir, showWarnings = TRUE)
#   print(paste0("created reports directory: ", ReportsDir))
# }
results_report_DF <- data.frame(Description=character(), ParameterValue=double())

ModelDatFilename<-paste0(RootDir, "/data/", CurPat, "/", CurPat, "_", CurTask, "_posdat.csv")
if(!file.exists(ModelDatFilename)){
  print("**ERROR** Input data file not found")
  print(sprintf("\tfile: ", ModelDatFilename))
  print(sprintf("\troot dir: ", RootDir))
}
PosDat<-read.csv(ModelDatFilename)

# may already be done in datafile
# if(RemoveFinalPosition){
#   PosDat <- PosDat[PosDat$pos < PosDat$stimlen,] # remove final position
# }
PosDat <- PosDat[PosDat$stimlen >= MinLength,] # include only lengths with sufficient N
PosDat <- PosDat[PosDat$stimlen <= MaxLength,]
# include only correct and nonword errors (not no response, word or w+nw errors)
PosDat <- PosDat[(PosDat$err_type == "correct") | (PosDat$err_type == "nonword"), ]
PosDat <- PosDat[(PosDat$pos) != (PosDat$stimlen), ]
# make sure final positions have been removed
PosDat$stim_number <- NumberStimuli(PosDat)
PosDat$raw_log_freq <- PosDat$log_freq
PosDat$log_freq <- PosDat$log_freq - mean(PosDat$log_freq) # centre frequency
OrigDataLen <- nrow(PosDat)
print(sprintf ("Original data has %d values", OrigDataLen))

## [1] "Original data has 4207 values"

if(RemoveFracPres){ # eliminate fractional preserved values?
  PosDat <- PosDat[(PosDat$preserved == 0) | (PosDat$preserved == 1),]
  DataLen<- nrow(PosDat)
  print(sprintf("Data without fractional preserved values has %d values", DataLen))
  print(sprintf("%d values eliminated.", OrigDataLen - DataLen))
}

```

```
## [1] "Data without fractional preserved values has 4137 values"
## [1] "70 values eliminated."
```

```
PosDat$CumPres <- CalcCumPres(PosDat)
PosDat$CumErr <- CalcCumErrFromPreserved(PosDat)
```

```
# plot distribution of complex onsets and codas
# across serial position in the stimuli -- do complexities concentrate
# on one part of the serial position curve?
# syll_component = 1 is coda
# syll_component = S is satellite
```

```
# get counts of syllable components in different serial positions
```

```
syll_comp_dist_N <- PosDat %>% mutate(pos_factor = as.factor(pos), syll_component_factor = as.factor(syll_component))
```

```
syll_comp_dist_perc <- syll_comp_dist_N %>% ungroup() %>%
  mutate(across(O:S, ~ . / total))
```

```
kable(syll_comp_dist_N)
```

pos_factor	O	P	V	1	S	total
1	527	29	128	NA	NA	684
2	61	NA	411	96	106	674
3	300	NA	160	201	14	675
4	291	NA	227	64	37	619
5	221	NA	202	71	32	526
6	201	1	123	69	21	415
7	166	NA	98	24	18	306
8	85	NA	48	24	4	161
9	71	NA	2	NA	4	77

```
kable(syll_comp_dist_perc)
```

pos_factor	O	P	V	1	S	total
1	0.7704678	0.0423977	0.1871345	NA	NA	684
2	0.0905045	NA	0.6097923	0.1424332	0.1572700	674
3	0.4444444	NA	0.2370370	0.2977778	0.0207407	675
4	0.4701131	NA	0.3667205	0.1033926	0.0597738	619
5	0.4201521	NA	0.3840304	0.1349810	0.0608365	526
6	0.4843373	0.0024096	0.2963855	0.1662651	0.0506024	415
7	0.5424837	NA	0.3202614	0.0784314	0.0588235	306
8	0.5279503	NA	0.2981366	0.1490683	0.0248447	161
9	0.9220779	NA	0.0259740	NA	0.0519481	77

```
# get percentages only from summary table
```

```
syll_comp_perc <- syll_comp_dist_perc[,2:(ncol(syll_comp_dist_perc)-1)]
```

```
syll_comp_perc$pos <- as.integer(as.character(syll_comp_dist_perc$pos_factor))
```

```
syll_comp_perc <- syll_comp_perc %>% rename("Coda" = "1", "Satellite" = "S")
```

```
# pivot to long format for plotting
```

```
syll_comp_plot_data <- syll_comp_perc %>% pivot_longer(cols=seq(1:(ncol(syll_comp_perc)-1)),
  names_to="syll_component", values_to="percent") %>% filter(syll_component %in% c("Coda", "Satellite"))
```

```
# plot satellite and coda occurrences across position
```

```
syll_comp_plot <- ggplot(syll_comp_plot_data,
  aes(x=pos,y=percent,group=syll_component,
    color=syll_component,
    linetype = syll_component,
    shape = syll_component)) +
  geom_line() + geom_point() + scale_color_manual(name="Syllable component", values = palette_values) +
syll_comp_plot <- syll_comp_plot + scale_y_continuous(name="Percent of segment types") + scale_linetype
  scale_shape_discrete(name="Syllable component")
ggsave(paste0(FigDir, CurPat, "_", RunLabel, "_", CurTask, "_pos_of_syll_complexities_in_stimuli.png"), plot=s
```

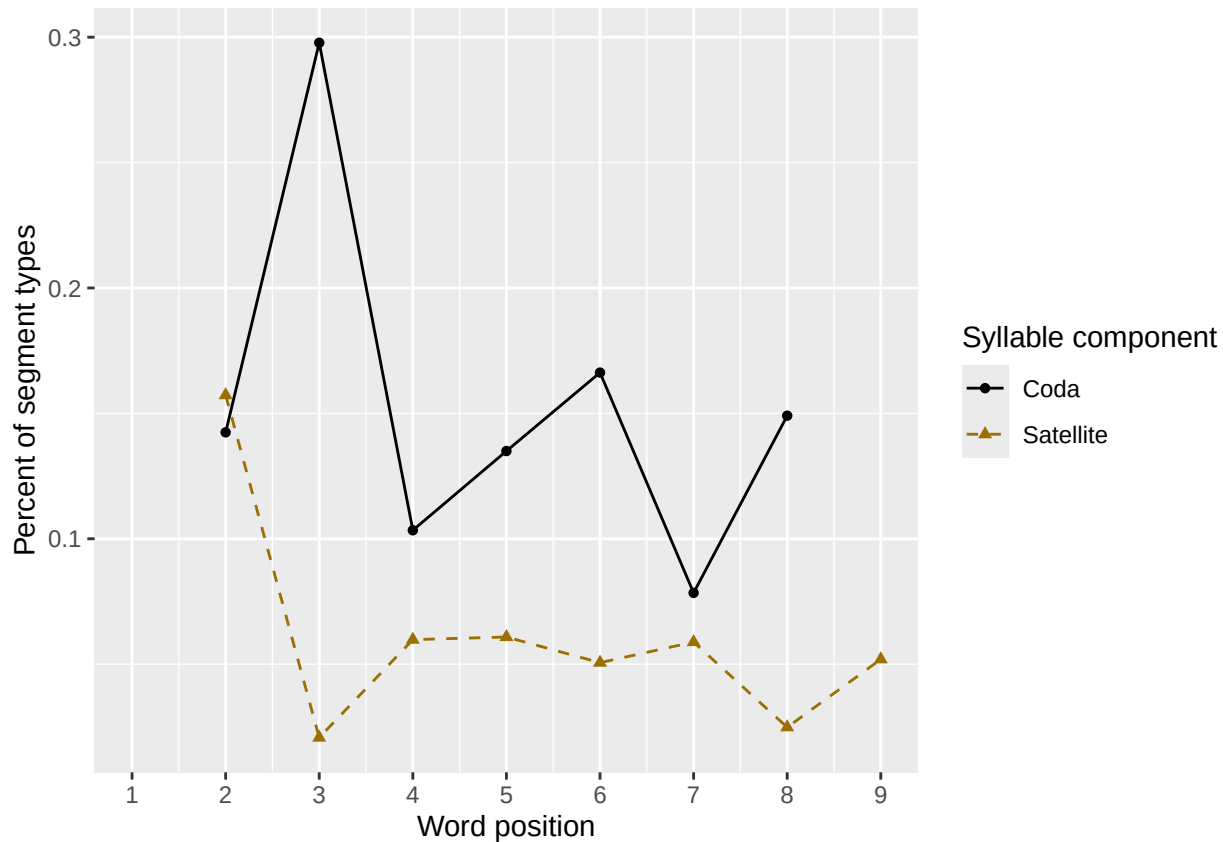
```
## Warning: Removed 3 rows containing missing values or values outside the scale range (`geom_line()`).
```

```
## Warning: Removed 3 rows containing missing values or values outside the scale range (`geom_point()`).
```

```
syll_comp_plot
```

```
## Warning: Removed 3 rows containing missing values or values outside the scale range (`geom_line()`).
```

```
## Removed 3 rows containing missing values or values outside the scale range (`geom_point()`).
```



```
# len/pos table
```

```
pos_len_summary <- PosDat %>% group_by(stimlen,pos) %>% summarise(preserved = mean(preserved))
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
min_preserved_raw <- min(as.numeric(unlist(pos_len_summary$preserved)))
```

```
max_preserved_raw <- max(as.numeric(unlist(pos_len_summary$preserved)))
```

```
preserved_range <- (max_preserved_raw - min_preserved_raw)
```

```

min_preserved <- min_preserved_raw - (0.1 * preserved_range)
max_preserved <- max_preserved_raw + (0.1 * preserved_range)
pos_len_table <- pos_len_summary %>% pivot_wider(names_from = pos, values_from = preserved)
write.csv(pos_len_table, paste0(TablesDir, CurPat, "_", RunLabel, "_", CurTask, "_preserved_percentages_by_len",
pos_len_table

```

```

## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`
##   <int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1      4 0.984 0.984 0.984 NA     NA     NA     NA     NA     NA
## 2      5 0.913 0.957 0.945 0.945 NA     NA     NA     NA     NA
## 3      6 0.982 0.964 0.946 0.955 0.920 NA     NA     NA     NA
## 4      7 0.964 0.936 0.917 0.927 0.936 0.927 NA     NA     NA
## 5      8 0.951 0.915 0.923 0.825 0.852 0.833 0.826 NA     NA
## 6      9 0.943 0.867 0.855 0.773 0.837 0.788 0.767 0.759 NA
## 7     10 0.961 0.933 0.923 0.893 0.857 0.829 0.816 0.851 0.831

```

```

# len/pos table
pos_len_N <- PosDat %>% group_by(stimlen, pos) %>% summarise(preserved = mean(preserved), N=n()) %>% dplyr::

```

`summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.

```

pos_len_N_table <- pos_len_N %>% pivot_wider(names_from = pos, values_from = N)
write.csv(pos_len_N_table, paste0(TablesDir, CurPat, "_", RunLabel, "_", CurTask,
"_N_by_len_position.csv"))
pos_len_N_table

```

```

## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`
##   <int> <int> <int> <int> <int> <int> <int> <int> <int> <int>
## 1      4    61    61    61    NA    NA    NA    NA    NA    NA
## 2      5    92    92    91    91    NA    NA    NA    NA    NA
## 3      6   112   112   111   112   112    NA    NA    NA    NA
## 4      7   110   109   108   110   109   110    NA    NA    NA
## 5      8   144   142   143   143   142   144   144    NA    NA
## 6      9    88    83    83    88    86    85    86    87    NA
## 7     10    77    75    78    75    77    76    76    74    77

```

```

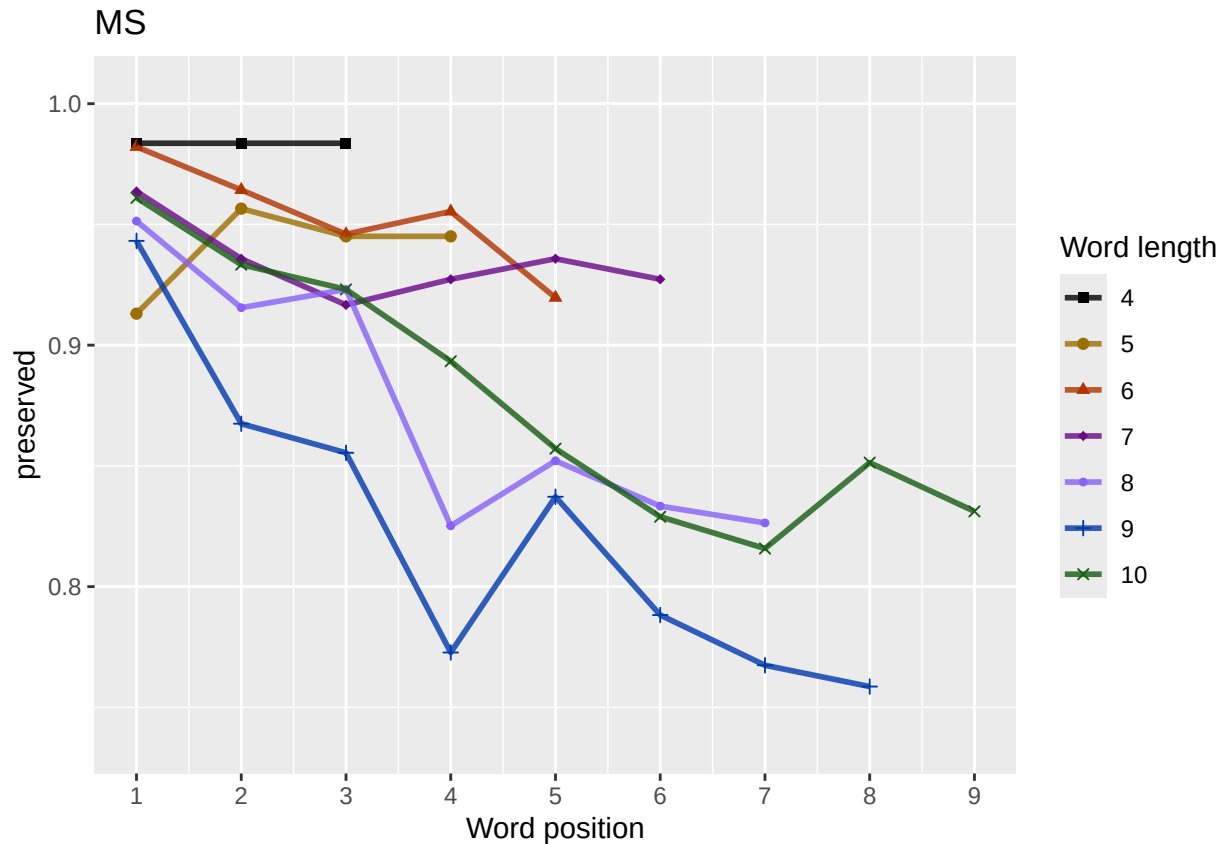
obs_linetypes <- c("solid", "solid", "solid", "solid",
"solid", "solid", "solid", "solid", "solid")
pred_linetypes <- "dashed"
pos_len_summary$stimlen <- factor(pos_len_summary$stimlen)
pos_len_summary$pos <- factor(pos_len_summary$pos)
# len_pos_plot <- ggplot(pos_len_summary, aes(x=pos, y=preserved, group=stimlen, shape=stimlen, color=stimlen))
len_pos_plot <- plot_len_pos_obs_predicted(PosDat,
paste0(CurPat),
NULL,
c(min_preserved, max_preserved),
palette_values,
shape_values,
obs_linetypes,
pred_linetypes)

```

`summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.
```

```
ggsave(paste0(FigDir, CurPat, "_", RunLabel, "_", CurTask,
              "_percent_preserved_by_length_pos.png"),
       plot=len_pos_plot, device="png", unit="cm", width=15, height=11)
len_pos_plot
```



Length and position

```
# length and position
```

```
LPMModelEquations<-c("preserved ~ 1",
  "preserved ~ stimlen",
  "preserved ~ pos",
  "preserved ~ stimlen + pos",
  "preserved ~ stimlen*pos",
  "preserved ~ I(pos^2)+pos",
  "preserved ~ stimlen + I(pos^2) + pos",
  "preserved ~ stimlen * (I(pos^2) + pos)"
)
```

```
LPres<-TestModels(LPMModelEquations,PosDat)
```

```
## *****
```

```
## model index: 8
```

```
##
```

```
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
```

```
## data = PosDat)
```

```

##
## Coefficients:
##      (Intercept)      stimlen      I(pos^2)      pos  stimlen:I(pos^2)      stiml
##      2.68851      0.06742      -0.14278      0.95855      0.01938      -0
##
## Degrees of Freedom: 4136 Total (i.e. Null); 4131 Residual
## Null Deviance:      2735
## Residual Deviance: 2604 AIC: 2616
## log likelihood: -1301.769
## Nagelkerke R2: 0.0645278
## % pres/err predicted correctly: -736.0286
## % of predictable range [ (model-null)/(1-null) ]: 0.03161273
## *****
## model index: 7
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
##      (Intercept)      stimlen      I(pos^2)      pos
##      4.97969      -0.20069      0.03044      -0.44942
##
## Degrees of Freedom: 4136 Total (i.e. Null); 4133 Residual
## Null Deviance:      2735
## Residual Deviance: 2609 AIC: 2617
## log likelihood: -1304.547
## Nagelkerke R2: 0.06183604
## % pres/err predicted correctly: -737.3501
## % of predictable range [ (model-null)/(1-null) ]: 0.0298764
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
##      (Intercept)      stimlen      pos  stimlen:pos
##      5.09424      -0.27265      -0.38372      0.02467
##
## Degrees of Freedom: 4136 Total (i.e. Null); 4133 Residual
## Null Deviance:      2735
## Residual Deviance: 2615 AIC: 2623
## log likelihood: -1307.648
## Nagelkerke R2: 0.05882693
## % pres/err predicted correctly: -738.2451
## % of predictable range [ (model-null)/(1-null) ]: 0.02870043
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
##      (Intercept)      stimlen      pos

```

```

##      4.3124      -0.1812      -0.1673
##
## Degrees of Freedom: 4136 Total (i.e. Null);  4134 Residual
## Null Deviance:      2735
## Residual Deviance: 2617  AIC: 2623
## log likelihood:  -1308.653
## Nagelkerke R2:  0.05785054
## % pres/err predicted correctly:  -738.3562
## % of predictable range [ (model-null)/(1-null) ]:  0.02855446
## *****
## model index:  6
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)          pos
##      3.48470      0.02035     -0.41839
##
## Degrees of Freedom: 4136 Total (i.e. Null);  4134 Residual
## Null Deviance:      2735
## Residual Deviance: 2638  AIC: 2644
## log likelihood:  -1318.914
## Nagelkerke R2:  0.04785687
## % pres/err predicted correctly:  -742.1662
## % of predictable range [ (model-null)/(1-null) ]:  0.02354854
## *****
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##      3.1250      -0.2258
##
## Degrees of Freedom: 4136 Total (i.e. Null);  4135 Residual
## Null Deviance:      2735
## Residual Deviance: 2642  AIC: 2646
## log likelihood:  -1320.796
## Nagelkerke R2:  0.04601865
## % pres/err predicted correctly:  -742.3609
## % of predictable range [ (model-null)/(1-null) ]:  0.02329276
## *****
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      4.3408      -0.2739
##
## Degrees of Freedom: 4136 Total (i.e. Null);  4135 Residual

```



```
## Null Deviance:      2735
## Residual Deviance: 2662 AIC: 2666
## log likelihood: -1331.111
## Nagelkerke R2: 0.03591386
## % pres/err predicted correctly: -747.9231
## % of predictable range [ (model-null)/(1-null) ]: 0.01598444
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
## 2.17
##
## Degrees of Freedom: 4136 Total (i.e. Null); 4136 Residual
## Null Deviance:      2735
## Residual Deviance: 2735 AIC: 2737
## log likelihood: -1367.358
## Nagelkerke R2: 0
## % pres/err predicted correctly: -760.0887
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****
```

```
BestLPModel<-LPRes$ModelResult[[1]]
BestLPModelFormula<-LPRes$Model[[1]]

LPAICSummary<-data.frame(Model=LPRes$Model,
                          AIC=LPRes$AIC,
                          row.names = LPRes$Model)
LPAICSummary$DeltaAIC<-LPAICSummary$AIC-LPAICSummary$AIC[1]
LPAICSummary$AICexp<-exp(-0.5*LPAICSummary$DeltaAIC)
LPAICSummary$AICwt<-LPAICSummary$AICexp/sum(LPAICSummary$AICexp)
LPAICSummary$NagR2<-LPRes$NagR2

LPAICSummary <- merge(LPAICSummary,LPRes$CoefficientValues, by='row.names',sort=FALSE)
LPAICSummary <- subset(LPAICSummary, select = -c(Row.names))

write.csv(LPAICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                              "_len_pos_model_summary.csv"),row.names = FALSE)

kable(LPAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2 (Intercept)	stimlen	pos	stimlen:pos	len:pos^2	stimlen:I(pos^2)
preserved ~ stimlen * (I(pos^2) + pos)	2615.533	0.000000	1.000000	0.066420	0.064528	885150.067423	1.9585529	-	-	0.0193764
								0.1606280	0.1427799	
preserved ~ stimlen + I(pos^2) + pos	2617.091	1.556013	0.459320	0.030610	0.061834	0.979689	-	-	NA	0.0304380
						0.200694	0.4494221			NA
preserved ~ stimlen * pos	2623.297	7.758250	0.020668	0.013770	0.058826	0.94240	-	-	0.0246682	NA
						0.272640	0.3837204			NA
preserved ~ stimlen + pos	2623.307	7.768735	0.020560	0.013702	0.057850	0.512373	-	-	NA	NA
						0.181248	0.1672667			NA

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:I(pos^2)	I(pos^2)	stimlen:I(pos^2)
preserved ~ I(pos^2) + pos	2643.8228	28.290815	0.000000	0.000000	0.50478569	484701	NA	-	NA	0.0203483	NA
preserved ~ pos	2645.5930	0.054545	0.000000	0.000000	0.20460187	125046	NA	-	NA	NA	NA
preserved ~ stimlen	2666.2210	0.683075	0.000000	0.000000	0.00359133	40788	-	NA	NA	NA	NA
preserved ~ 1	2736.7162	1.178408	0.000000	0.000000	0.000000	00169862	NA	NA	NA	NA	NA

```
print(BestLPModelFormula)
```

```
## [1] "preserved ~ stimlen * (I(pos^2) + pos)"
```

```
print(BestLPModel)
```

```
##
```

```
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
```

```
## data = PosDat)
```

```
##
```

```
## Coefficients:
```

```
## (Intercept) stimlen I(pos^2) pos stimlen:I(pos^2) stimlen:I(pos^2)
```

```
## 2.68851 0.06742 -0.14278 0.95855 0.01938 -0.01938
```

```
##
```

```
## Degrees of Freedom: 4136 Total (i.e. Null); 4131 Residual
```

```
## Null Deviance: 2735
```

```
## Residual Deviance: 2604 AIC: 2616
```

```
PosDat$LPFitted<-fitted(BestLPModel)
```

```
fitted_len_pos_plot <- plot_len_pos_obs_predicted(PosDat, PosDat$patient[1],
NULL,palette_values=palette_values)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
# len/pos table
```

```
fitted_pos_len_summary <- PosDat %>% group_by(stimlen,pos) %>% summarise(fitted = mean(LPfitted))
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
fitted_pos_len_table <- fitted_pos_len_summary %>% pivot_wider(names_from = pos, values_from = fitted)
```

```
fitted_pos_len_table
```

```
## # A tibble: 7 x 10
```

```
## # Groups: stimlen [7]
```

```
## stimlen `1` `2` `3` `4` `5` `6` `7` `8` `9`
```

```
## <int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
```

```
## 1 4 0.961 0.965 0.965 NA NA NA NA NA NA
```

```
## 2 5 0.958 0.959 0.956 0.948 NA NA NA NA NA
```

```
## 3 6 0.955 0.952 0.945 0.934 0.917 NA NA NA NA
```

```
## 4 7 0.952 0.943 0.931 0.916 0.896 0.871 NA NA NA
```

```
## 5 8 0.949 0.932 0.914 0.893 0.870 0.847 0.824 NA NA
```

```
## 6 9 0.945 0.920 0.893 0.864 0.839 0.819 0.808 0.806 NA
```

```
## 7 10 0.941 0.906 0.867 0.830 0.802 0.788 0.790 0.809 0.841
```

```
fitted_pos_len_summary$stimlen<-factor(fitted_pos_len_summary$stimlen)
```

```
fitted_pos_len_summary$pos<-factor(fitted_pos_len_summary$pos)
```

```

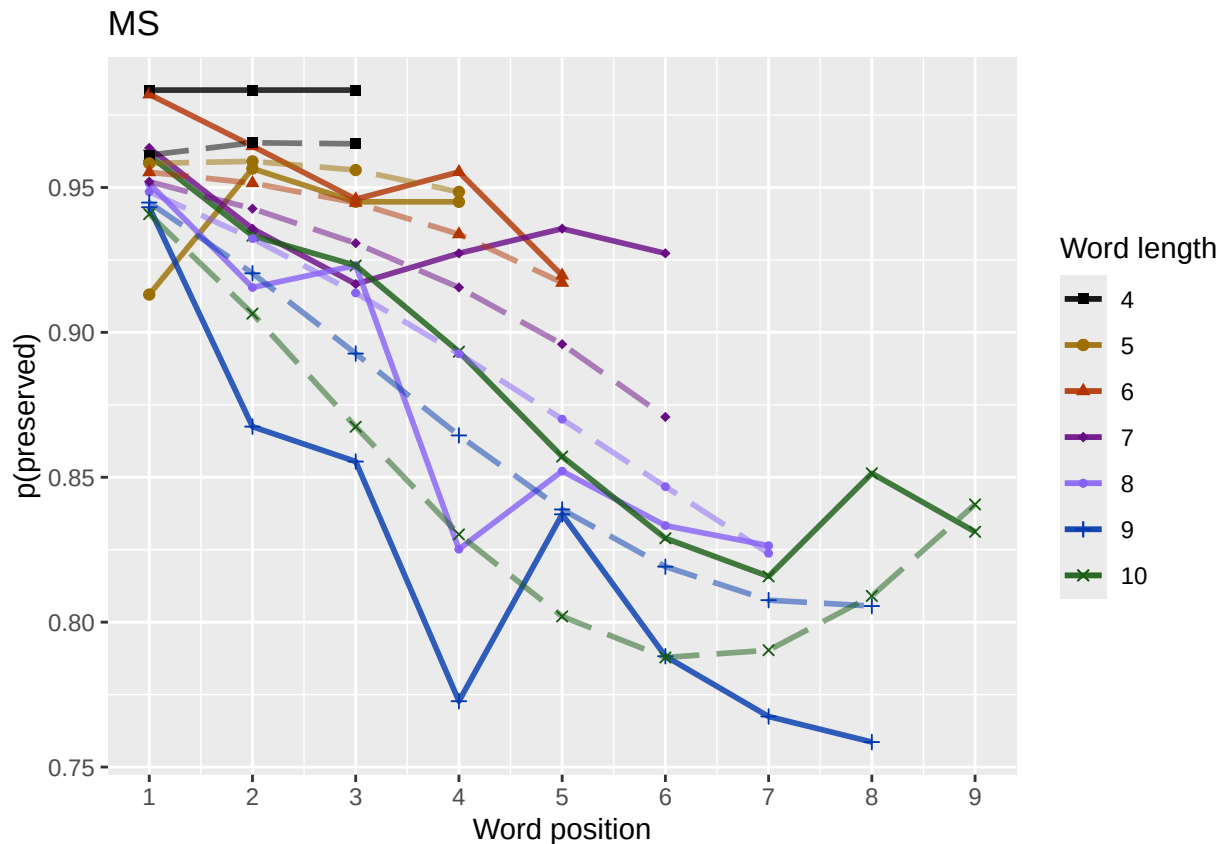
# fitted_len_pos_plot <- ggplot(pos_len_summary, aes(x=pos, y=preserved, group=stimlen, shape=stimlen, color=
# fitted_len_pos_plot <- fitted_len_pos_plot + geom_line(data=fitted_pos_len_summary, aes(x=pos, y=fitted
# geom_point(data=fitted_pos_len_summary, aes(x=pos, y=fitted, group=stimlen, shape=stimlen))) + ggtitle(pas

fitted_len_pos_plot <- plot_len_pos_obs_predicted(PosDat,
  paste0(PosDat$patient[1]),
  "LPFitted",
  NULL,
  palette_values,
  shape_values,
  obs_linetypes,
  pred_linetypes = c("longdash")
)

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.

ggsave(paste0(FigDir, CurPat, "_", RunLabel, "_", CurTask, "_percent_preserved_by_length_pos_wfit.png"), plot=fitted_len_pos_plot

```



length and position without fragments to see if this changes position² influence

```

# first number responses, then count resp with fragments - below we will eliminate fragments
# and re-run models

# number responses
resp_num <- 0

```

```

prev_pos<-9999 # big number to initialize (so first position is smaller)
resp_num_array <- integer(length = nrow(PosDat))
for(i in seq(1,nrow(PosDat))){
  if(PosDat$pos[i] <= prev_pos){ # equal for resp length 1
    resp_num <- resp_num + 1
  }
  resp_num_array[i]<-resp_num
  prev_pos <- PosDat$pos[i]
}
PosDat$resp_num <- resp_num_array

# count responses with fragments
resp_with_frag <- PosDat %>% group_by(resp_num) %>% summarise(frag = as.logical(sum(fragment)))
num_frag <- resp_with_frag %>% summarise(frag_sum = sum(frag), N=n())
num_frag

## # A tibble: 1 x 2
##   frag_sum      N
##   <int> <int>
## 1      64   688

num_frag$percent_with_frag[1] <- (num_frag$frag_sum[1] / num_frag$N[1])*100

## Warning: Unknown or uninitialised column: `percent_with_frag`.

print(sprintf("The number of responses with fragments was %d / %d = %.2f percent",
  num_frag$frag_sum[1],num_frag$N[1],num_frag$percent_with_frag[1]))

## [1] "The number of responses with fragments was 64 / 688 = 9.30 percent"

write.csv(num_frag,paste0(TablesDir,CurPat,"_",RunLabel,"_",
  CurTask,"_percent_fragments.csv"),row.names = FALSE)

# length and position models for data without fragments
NoFragData <- PosDat %>% filter(fragment != 1)

NoFrag_LPRes<-TestModels(LPModelEquations,NoFragData)

## *****
## model index: 8
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##   data = PosDat)
##
## Coefficients:
##   (Intercept)      stimlen      I(pos^2)      pos  stimlen:I(pos^2)      stimlen
##   2.49934      0.09729      -0.14251      1.29540      0.02297      -0
##
## Degrees of Freedom: 3920 Total (i.e. Null); 3915 Residual
## Null Deviance: 1626
## Residual Deviance: 1589 AIC: 1601
## log likelihood: -794.2539
## Nagelkerke R2: 0.02830873
## % pres/err predicted correctly: -389.2842
## % of predictable range [ (model-null)/(1-null) ]: 0.009260447
## *****

```

```

## model index: 7
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      I(pos^2)          pos
##      5.27298      -0.24572      0.05054      -0.36887
##
## Degrees of Freedom: 3920 Total (i.e. Null); 3917 Residual
## Null Deviance:      1626
## Residual Deviance: 1593 AIC: 1601
## log likelihood: -796.6421
## Nagelkerke R2: 0.02475322
## % pres/err predicted correctly: -389.66
## % of predictable range [ (model-null)/(1-null) ]: 0.008306294
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos
##      4.4402      -0.2235      0.0516
##
## Degrees of Freedom: 3920 Total (i.e. Null); 3918 Residual
## Null Deviance:      1626
## Residual Deviance: 1603 AIC: 1609
## log likelihood: -801.4862
## Nagelkerke R2: 0.01752792
## % pres/err predicted correctly: -390.8044
## % of predictable range [ (model-null)/(1-null) ]: 0.005401354
## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      4.4540      -0.2007
##
## Degrees of Freedom: 3920 Total (i.e. Null); 3919 Residual
## Null Deviance:      1626
## Residual Deviance: 1605 AIC: 1609
## log likelihood: -802.5955
## Nagelkerke R2: 0.01587087
## % pres/err predicted correctly: -391.0196
## % of predictable range [ (model-null)/(1-null) ]: 0.004854992
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",

```

```

##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos  stimlen:pos
##      4.75240      -0.26020      -0.05180      0.01188
##
## Degrees of Freedom: 3920 Total (i.e. Null);  3917 Residual
## Null Deviance:      1626
## Residual Deviance: 1603  AIC: 1611
## log likelihood:  -801.3683
## Nagelkerke R2:  0.01770399
## % pres/err predicted correctly:  -390.7712
## % of predictable range [ (model-null)/(1-null) ]:  0.005485675
## *****
## model index:  6
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)          pos
##      3.40877      0.03724      -0.32079
##
## Degrees of Freedom: 3920 Total (i.e. Null);  3918 Residual
## Null Deviance:      1626
## Residual Deviance: 1621  AIC: 1627
## log likelihood:  -810.5015
## Nagelkerke R2:  0.004033461
## % pres/err predicted correctly:  -392.4213
## % of predictable range [ (model-null)/(1-null) ]:  0.001296867
## *****
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)
##      2.882
##
## Degrees of Freedom: 3920 Total (i.e. Null);  3920 Residual
## Null Deviance:      1626
## Residual Deviance: 1626  AIC: 1628
## log likelihood:  -813.1881
## Nagelkerke R2:  3.269993e-16
## % pres/err predicted correctly:  -392.9322
## % of predictable range [ (model-null)/(1-null) ]:  0
## *****
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:

```

```
## (Intercept)          pos
##      2.913411      -0.008424
##
## Degrees of Freedom: 3920 Total (i.e. Null);  3919 Residual
## Null Deviance:      1626
## Residual Deviance: 1626 AIC: 1630
## log likelihood:  -813.1572
## Nagelkerke R2:   4.640123e-05
## % pres/err predicted correctly:  -392.9264
## % of predictable range [ (model-null)/(1-null) ]:  1.459198e-05
## *****
```

```
NoFragBestLPModel<-NoFrag_LPRes$ModelResult[[1]]
NoFragBestLPModelFormula<-NoFrag_LPRes$Model[[1]]

NoFragLPAICSummary<-data.frame(Model=NoFrag_LPRes$Model,
                                AIC=NoFrag_LPRes$AIC,
                                row.names = NoFrag_LPRes$Model)
NoFragLPAICSummary$DeltaAIC<-NoFragLPAICSummary$AIC-NoFragLPAICSummary$AIC[1]
NoFragLPAICSummary$AICexp<-exp(-0.5*NoFragLPAICSummary$DeltaAIC)
NoFragLPAICSummary$AICwt<-NoFragLPAICSummary$AICexp/sum(NoFragLPAICSummary$AICexp)
NoFragLPAICSummary$NagR2<-NoFrag_LPRes$NagR2

NoFragLPAICSummary <- merge(NoFragLPAICSummary,NoFrag_LPRes$CoefficientValues, by='row.names',sort=FALSE)
NoFragLPAICSummary <- subset(NoFragLPAICSummary, select = -c(Row.names))

write.csv(NoFragLPAICSummary,paste0(TablesDir,
                                    CurPat,"_",RunLabel,"_",
                                    CurTask,
                                    "_no_fragments_len_pos_model_summary.csv"),
          row.names = FALSE)
kable(NoFragLPAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:I(pos)	stimlen:I(pos^2)	stimlen:I(pos^2)
preserved ~ stimlen * (I(pos^2) + pos)	1600.508	0.000000	1.000000	0.005841605	0.0283087	4993350.097294	102954043	-	-	0.022972	
preserved ~ stimlen + I(pos^2) + pos	1601.284	0.776303	0.678309	0.063962407	0.0247532	272979	-	-	NA	0.0505430	NA
preserved ~ stimlen + pos	1608.972	8.464530	0.0145194	0.0084807	0.175279	440198	-	0.0516042	NA	NA	NA
preserved ~ stimlen	1609.198	8.683072	0.0130165	0.0076037	0.158709	453964	-	NA	NA	NA	NA
preserved ~ stimlen * pos	1610.737	10.228702	0.0060098	0.0035107	0.177040	752402	-	-	0.011880	NA	NA
preserved ~ I(pos^2) + pos	1627.002	26.495089	0.000000	0.000000	0.0004033	5408767	NA	-	NA	0.0372398	NA
preserved ~ 1	1628.372	27.868322	0.000000	0.000000	0.000000	0.0082057	NA	NA	NA	NA	NA
preserved ~ pos	1630.312	29.806550	0.000000	0.000000	0.0000424	913411	NA	-	NA	NA	NA

```
# plot no fragment data
```

```
NoFragData$LPFitted<-fitted(NoFragBestLPModel)
nofrag_obs_len_pos_plot <- plot_len_pos_obs_predicted(NoFragData, NoFragData$patient[1],
  NULL,palette_values=palette_values)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
# len/pos table
```

```
nofrag_fitted_pos_len_summary <- NoFragData %>% group_by(stimlen,pos) %>% summarise(fitted = mean(LPfit
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
nofrag_fitted_pos_len_table <- nofrag_fitted_pos_len_summary %>% pivot_wider(names_from = pos, values_f
nofrag_fitted_pos_len_table
```

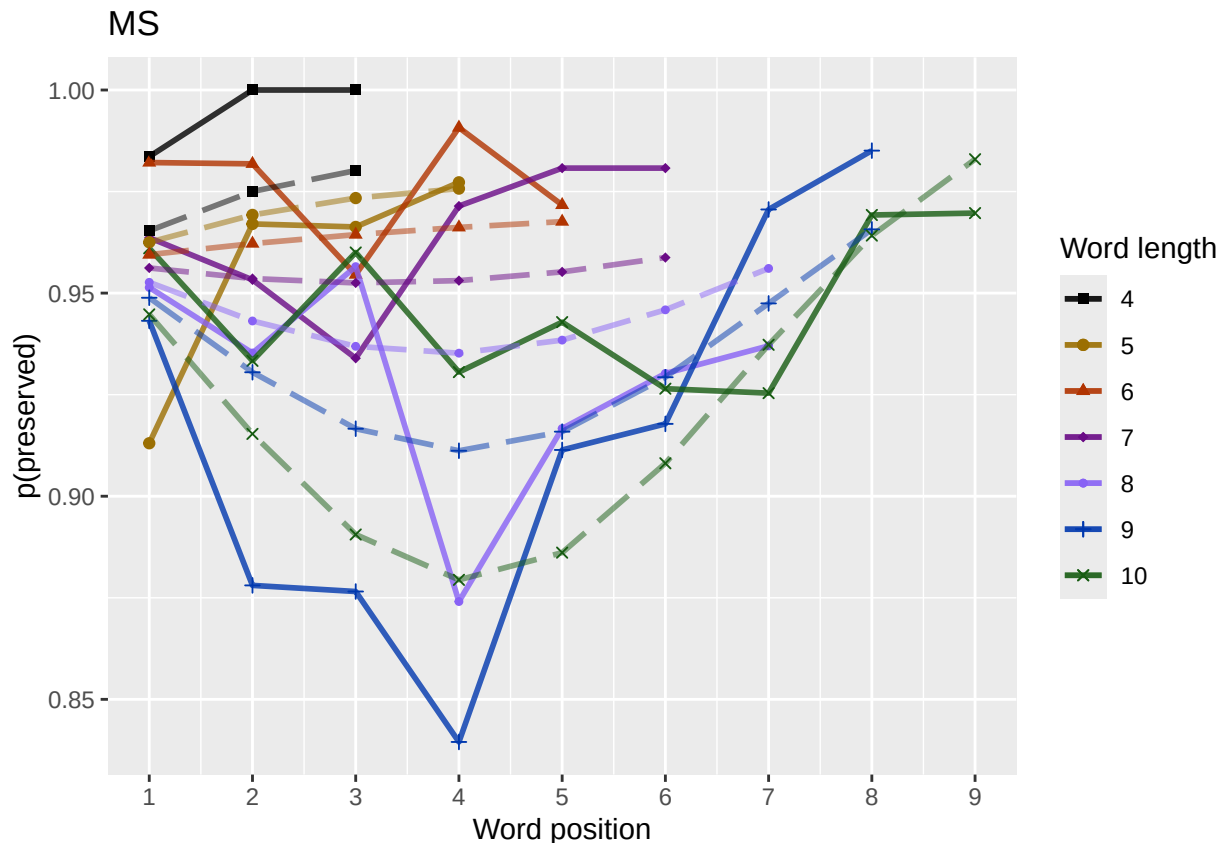
```
## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`
##   <int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1      4 0.965 0.975 0.980 NA      NA      NA      NA      NA      NA
## 2      5 0.963 0.969 0.973 0.976 NA      NA      NA      NA      NA
## 3      6 0.959 0.962 0.964 0.966 0.968 NA      NA      NA      NA
## 4      7 0.956 0.954 0.953 0.953 0.955 0.959 NA      NA      NA
## 5      8 0.953 0.943 0.937 0.935 0.938 0.946 0.956 NA      NA
## 6      9 0.949 0.931 0.917 0.911 0.916 0.929 0.947 0.966 NA
## 7     10 0.945 0.915 0.891 0.879 0.886 0.908 0.937 0.964 0.983
```

```
fitted_pos_len_summary$stimlen<-factor(nofrag_fitted_pos_len_summary$stimlen)
fitted_pos_len_summary$pos<-factor(nofrag_fitted_pos_len_summary$pos)
# fitted_len_pos_plot <- ggplot(pos_len_summary,aes(x=pos,y=preserved,group=stimlen,shape=stimlen,color=stimlen))
# fitted_len_pos_plot <- fitted_len_pos_plot + geom_line(data=fitted_pos_len_summary, aes(x=pos,y=fitted,group=stimlen,shape=stimlen))
# geom_point(data=fitted_pos_len_summary,aes(x=pos,y=fitted,group=stimlen,shape=stimlen)) + ggtitle(paste0("no_fragments_percent_preserved_by_length_pos_wfit.png"))
```

```
nofrag_fitted_len_pos_plot <- plot_len_pos_obs_predicted(NoFragData,
  paste0(NoFragData$patient[1]),
  "LPFitted",
  NULL,
  palette_values,
  shape_values,
  obs_linetypes,
  pred_linetypes = c("longdash")
)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
ggsave(paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,
  "no_fragments_percent_preserved_by_length_pos_wfit.png"),
  plot=nofrag_fitted_len_pos_plot,
  device="png",unit="cm",
  width=15,height=11)
nofrag_fitted_len_pos_plot
```

back to full data

```
results_report_DF <- AddReportLine(results_report_DF,"min preserved",min_preserved)
results_report_DF <- AddReportLine(results_report_DF,"max preserved",max_preserved)
print(sprintf("Min/max preserved range: %.2f - %.2f",min_preserved,max_preserved))
```

```
## [1] "Min/max preserved range: 0.74 - 1.01"
```

```
# find the average difference between lengths and the average difference
# between positions taking the other factor into account
```

```
# choose fitted or raw -- we use fitted values because they are smoothed averages
# that take into account the influence of the other variable
```

```
table_to_use <- fitted_pos_len_table
```

```
# take away column with length information
```

```
table_to_use <- table_to_use[,2:ncol(table_to_use)]
```

```
# take averages for positions
```

```
# don't want downward estimates influenced by return upward of U
```

```
# therefore, for downward influence, use only the values before the min
```

```
# take the difference between each value (differences between position proportion correct) **NOTE** pro
```

```
# average the difference in probabilities
```

```
# in case min is close to the end or we are not using a min (for non-U-shaped)
```

```
# functions, we need to get differences _first_ and then average those (e.g.
```

```
# if there is an effect of length and we just average position data,
```

```
# later positions) will have a lower average (because of the length effect)
```

```
# even if all position functions go upward
```

```

table_pos_diffs <- t(diff(t(as.matrix(table_to_use))))
ncol_pos_diff_matrix <- ncol(table_pos_diffs)
first_col_mean <- mean(as.numeric(unlist(table_pos_diffs[,1])))
potential_u_shape <- 0
if(first_col_mean < 0){ # downward, so potential U-shape
  # take only negative values from the table
  filtered_table_pos_diffs<-table_pos_diffs
  filtered_table_pos_diffs[table_pos_diffs >= 0] <- NA
  potential_u_shape <- 1
}else{
  # upward - use the positive values
  # take only negative values from the table
  filtered_table_pos_diffs<-table_pos_diffs
  filtered_table_pos_diffs[table_pos_diffs <= 0] <- NA
}
average_pos_diffs <- apply(filtered_table_pos_diffs,2,mean,na.rm=TRUE)
OA_mean_pos_diff <- mean(average_pos_diffs,na.rm=TRUE)

table_len_diffs <- diff(as.matrix(table_to_use))
average_len_diffs <- apply(table_len_diffs,1,mean,na.rm=TRUE)
OA_mean_len_diff <- mean(average_len_diffs,na.rm=TRUE)
CurrentLabel<-"mean change in probability for each additional length: "
print(CurrentLabel)

## [1] "mean change in probability for each additional length: "
print(OA_mean_len_diff)

## [1] -0.0142527
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,OA_mean_len_diff)

CurrentLabel<-"mean change in probability for each additional position (excluding U-shape): "
print(CurrentLabel)

## [1] "mean change in probability for each additional position (excluding U-shape): "
print(OA_mean_pos_diff)

## [1] -0.01648688
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,OA_mean_pos_diff)

if(potential_u_shape){ # downward, so potential U-shape
  # take only positive values from the table
  filtered_pos_upward_u<-table_pos_diffs
  filtered_pos_upward_u[table_pos_diffs <= 0] <- NA
  average_pos_u_diffs <- apply(filtered_pos_upward_u,
                              2,mean,na.rm=TRUE)
  OA_mean_pos_u_diff <- mean(average_pos_u_diffs,na.rm=TRUE)
  if(is.nan(OA_mean_pos_u_diff) | (OA_mean_pos_u_diff <= 0)){
    print("No U-shape in this participant")
    results_report_DF <- AddReportLine(results_report_DF, "U-shape", 0)
    potential_u_shape <- FALSE
  }else{
    results_report_DF <- AddReportLine(results_report_DF, "U-shape", 1)
  }
}

```

```

    CurrentLabel<-"Average upward change after U minimum"
    print(CurrentLabel)
    print(OA_mean_pos_u_diff)
    results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, OA_mean_pos_u_diff)

    CurrentLabel<-"Proportion of average downward change"
    prop_ave_down <- abs(OA_mean_pos_u_diff/OA_mean_pos_diff)
    results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, prop_ave_down)
  }
}else{
  print("No U-shape in this participant")
  results_report_DF <- AddReportLine(results_report_DF, "U-shape", 0)
}

## [1] "Average upward change after U minimum"
## [1] 0.0138211

if(potential_u_shape){
  n_rows<-nrow(table_to_use)
  downward_dist <- numeric(n_rows)
  upward_dist <- numeric(n_rows)
  for(i in seq(1,n_rows)){
    current_row <- as.numeric(unlist(table_to_use[i,1:9]))
    current_row <- current_row[!is.na(current_row)]
    current_row_len <- length(current_row)
    row_min <- min(current_row,na.rm=TRUE)
    min_pos <- which(current_row == row_min)
    left_max <- max(current_row[1:min_pos],na.rm=TRUE)
    right_max <- max(current_row[min_pos:current_row_len])
    left_diff <- left_max - row_min
    right_diff <- right_max - row_min
    downward_dist[i]<-left_diff
    upward_dist[i]<-right_diff
  }
  print("differences from left max to min for each row: ")
  print(downward_dist)
  print("differences from min to right max for each row: ")
  print(upward_dist)

  # Use the max return upward (min of upward_dist) and then the
  # downward distance from the left for the same row

  print(" ")
  biggest_return_upward_row <- which(upward_dist == max(upward_dist))
  CurrentLabel <- "row with biggest return upward"
  print(CurrentLabel)
  print(biggest_return_upward_row)
  results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, biggest_return_upward_row)

  print(" ")
  CurrentLabel<-"downward distance for row with the largest upward value"
  print(CurrentLabel)
  print(downward_dist[biggest_return_upward_row])
  results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, downward_dist[biggest_return_upward_row])
}

```

```

CurrentLabel<-"return upward value"
print(upward_dist[biggest_return_upward_row])
results_report_DF <- AddReportLine(results_report_DF,
                                   CurrentLabel,
                                   upward_dist[biggest_return_upward_row])

print(" ")
# percentage return upward
percentage_return_upward <- upward_dist[biggest_return_upward_row]/downward_dist[biggest_return_upward_row]
CurrentLabel <- "Return upward as a proportion of the downward distance:"
print(CurrentLabel)
print(percentage_return_upward)
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,
                                   percentage_return_upward)
}else{
  print("no U-shape in this participant")
}

## [1] "differences from left max to min for each row: "
## [1] 0.00000000 0.01054045 0.03814210 0.08119140 0.12477524 0.13923479 0.15299868
## [1] "differences from min to right max for each row: "
## [1] 0.004246177 0.000000000 0.000000000 0.000000000 0.000000000 0.000000000 0.052810313
## [1] " "
## [1] "row with biggest return upward"
## [1] 7
## [1] " "
## [1] "downward distance for row with the largest upward value"
## [1] 0.1529987
## [1] 0.05281031
## [1] " "
## [1] "Return upward as a proportion of the downward distance:"
## [1] 0.3451684

FLPModelEquations<-c(
  # models with log frequency, but no three-way interactions (due to difficulty interpreting and small
  "preserved ~ stimlen*log_freq",
  "preserved ~ stimlen+log_freq",
  "preserved ~ pos*log_freq",
  "preserved ~ pos+log_freq",
  "preserved ~ stimlen*log_freq + pos*log_freq",
  "preserved ~ stimlen*log_freq + pos",
  "preserved ~ stimlen + pos*log_freq",
  "preserved ~ stimlen + pos + log_freq",
  "preserved ~ (I(pos^2)+pos)*log_freq",
  "preserved ~ stimlen*log_freq + (I(pos^2) + pos)*log_freq",
  "preserved ~ stimlen*log_freq + I(pos^2) + pos",
  "preserved ~ stimlen + (I(pos^2) + pos)*log_freq",
  "preserved ~ stimlen + I(pos^2) + pos + log_freq",

  # models without frequency
  "preserved ~ 1",
  "preserved ~ stimlen",
  "preserved ~ pos",
  "preserved ~ stimlen + pos",

```

```

      "preserved ~ stimlen*pos",
      "preserved ~ I(pos^2)+pos",
      "preserved ~ stimlen + I(pos^2) + pos",
      "preserved ~ stimlen * (I(pos^2) + pos)"
    )

FLPRes<-TestModels(FLPModelEquations,PosDat)

## *****
## model index:  11
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##
## Coefficients:
##      (Intercept)      stimlen      log_freq      I(pos^2)      pos  stimlen:log
##           4.74746       -0.17555         0.50529         0.02880       -0.43411         -0
##
## Degrees of Freedom: 4136 Total (i.e. Null);  4131 Residual
## Null Deviance:      2735
## Residual Deviance: 2591  AIC: 2603
## log likelihood:  -1295.501
## Nagelkerke R2:   0.07058787
## % pres/err predicted correctly:  -734.1809
## % of predictable range [ (model-null)/(1-null) ]:  0.03404048
## *****
## model index:  10
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##
## Coefficients:
##      (Intercept)      stimlen      log_freq      I(pos^2)      pos  stim
##           4.720361       -0.179783         0.348971         0.026804       -0.408917
##      log_freq:pos
##           0.084402
##
## Degrees of Freedom: 4136 Total (i.e. Null);  4129 Residual
## Null Deviance:      2735
## Residual Deviance: 2588  AIC: 2604
## log likelihood:  -1293.909
## Nagelkerke R2:   0.07212402
## % pres/err predicted correctly:  -733.4812
## % of predictable range [ (model-null)/(1-null) ]:  0.03495987
## *****
## model index:  13
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##
## Coefficients:
##      (Intercept)      stimlen      I(pos^2)      pos      log_freq
##           4.76614       -0.17165         0.03016       -0.44716         0.09384
##

```

```

## Degrees of Freedom: 4136 Total (i.e. Null); 4132 Residual
## Null Deviance: 2735
## Residual Deviance: 2598 AIC: 2608
## log likelihood: -1299.074
## Nagelkerke R2: 0.06713602
## % pres/err predicted correctly: -735.3611
## % of predictable range [ (model-null)/(1-null) ]: 0.03248984
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen log_freq pos stimlen:log_freq
## 4.11259 -0.15661 0.51958 -0.16754 -0.05161
##
## Degrees of Freedom: 4136 Total (i.e. Null); 4132 Residual
## Null Deviance: 2735
## Residual Deviance: 2598 AIC: 2608
## log likelihood: -1299.146
## Nagelkerke R2: 0.06706632
## % pres/err predicted correctly: -735.2028
## % of predictable range [ (model-null)/(1-null) ]: 0.03269785
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen log_freq pos stimlen:log_freq log_fr
## 4.11386 -0.16112 0.52639 -0.15966 -0.06121 0
##
## Degrees of Freedom: 4136 Total (i.e. Null); 4131 Residual
## Null Deviance: 2735
## Residual Deviance: 2597 AIC: 2609
## log likelihood: -1298.506
## Nagelkerke R2: 0.06768471
## % pres/err predicted correctly: -734.9046
## % of predictable range [ (model-null)/(1-null) ]: 0.03308965
## *****
## model index: 12
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen I(pos^2) pos log_freq I(pos
## 4.708988 -0.171508 0.026212 -0.412755 -0.082375
##
## Degrees of Freedom: 4136 Total (i.e. Null); 4130 Residual
## Null Deviance: 2735
## Residual Deviance: 2595 AIC: 2609

```

```

## log likelihood: -1297.641
## Nagelkerke R2: 0.06852069
## % pres/err predicted correctly: -734.7422
## % of predictable range [ (model-null)/(1-null) ]: 0.03330296
## *****
## model index: 8
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos      log_freq
##      4.10295      -0.15200      -0.16780      0.09466
##
## Degrees of Freedom: 4136 Total (i.e. Null); 4133 Residual
## Null Deviance: 2735
## Residual Deviance: 2606 AIC: 2614
## log likelihood: -1303.084
## Nagelkerke R2: 0.06325403
## % pres/err predicted correctly: -736.4022
## % of predictable range [ (model-null)/(1-null) ]: 0.03112182
## *****
## model index: 21
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
##      (Intercept)      stimlen      I(pos^2)          pos stimlen:I(pos^2)      stiml
##      2.68851      0.06742      -0.14278      0.95855      0.01938      -0
##
## Degrees of Freedom: 4136 Total (i.e. Null); 4131 Residual
## Null Deviance: 2735
## Residual Deviance: 2604 AIC: 2616
## log likelihood: -1301.769
## Nagelkerke R2: 0.0645278
## % pres/err predicted correctly: -736.0286
## % of predictable range [ (model-null)/(1-null) ]: 0.03161273
## *****
## model index: 7
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos      log_freq pos:log_freq
##      4.10151      -0.15128      -0.16890      0.10336      -0.00187
##
## Degrees of Freedom: 4136 Total (i.e. Null); 4132 Residual
## Null Deviance: 2735
## Residual Deviance: 2606 AIC: 2616
## log likelihood: -1303.073
## Nagelkerke R2: 0.06326512
## % pres/err predicted correctly: -736.4131

```

```

## % of predictable range [ (model-null)/(1-null) ]: 0.0311075
## *****
## model index: 20
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      I(pos^2)          pos
##    4.97969    -0.20069     0.03044    -0.44942
##
## Degrees of Freedom: 4136 Total (i.e. Null); 4133 Residual
## Null Deviance:      2735
## Residual Deviance: 2609 AIC: 2617
## log likelihood: -1304.547
## Nagelkerke R2: 0.06183604
## % pres/err predicted correctly: -737.3501
## % of predictable range [ (model-null)/(1-null) ]: 0.0298764
## *****
## model index: 18
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos stimlen:pos
##    5.09424    -0.27265    -0.38372     0.02467
##
## Degrees of Freedom: 4136 Total (i.e. Null); 4133 Residual
## Null Deviance:      2735
## Residual Deviance: 2615 AIC: 2623
## log likelihood: -1307.648
## Nagelkerke R2: 0.05882693
## % pres/err predicted correctly: -738.2451
## % of predictable range [ (model-null)/(1-null) ]: 0.02870043
## *****
## model index: 17
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos
##    4.3124    -0.1812    -0.1673
##
## Degrees of Freedom: 4136 Total (i.e. Null); 4134 Residual
## Null Deviance:      2735
## Residual Deviance: 2617 AIC: 2623
## log likelihood: -1308.653
## Nagelkerke R2: 0.05785054
## % pres/err predicted correctly: -738.3562
## % of predictable range [ (model-null)/(1-null) ]: 0.02855446
## *****
## model index: 9

```



```

##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) I(pos^2) pos log_freq I(pos^2):log_freq
## 3.425111 0.017184 -0.381895 -0.022563 -0.009869
##
## Degrees of Freedom: 4136 Total (i.e. Null); 4131 Residual
## Null Deviance: 2735
## Residual Deviance: 2614 AIC: 2626
## log likelihood: -1307.231
## Nagelkerke R2: 0.05923221
## % pres/err predicted correctly: -737.6129
## % of predictable range [ (model-null)/(1-null) ]: 0.02953108
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) pos log_freq
## 3.0953 -0.2130 0.1216
##
## Degrees of Freedom: 4136 Total (i.e. Null); 4134 Residual
## Null Deviance: 2735
## Residual Deviance: 2622 AIC: 2628
## log likelihood: -1311.053
## Nagelkerke R2: 0.05551788
## % pres/err predicted correctly: -738.7434
## % of predictable range [ (model-null)/(1-null) ]: 0.02804577
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) pos log_freq pos:log_freq
## 3.110316 -0.217013 0.158946 -0.008157
##
## Degrees of Freedom: 4136 Total (i.e. Null); 4133 Residual
## Null Deviance: 2735
## Residual Deviance: 2622 AIC: 2630
## log likelihood: -1310.837
## Nagelkerke R2: 0.05572761
## % pres/err predicted correctly: -738.7318
## % of predictable range [ (model-null)/(1-null) ]: 0.02806099
## *****
## model index: 19
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)

```

```

##
## Coefficients:
## (Intercept)      I(pos^2)          pos
##      3.48470      0.02035      -0.41839
##
## Degrees of Freedom: 4136 Total (i.e. Null);  4134 Residual
## Null Deviance:      2735
## Residual Deviance: 2638  AIC: 2644
## log likelihood:  -1318.914
## Nagelkerke R2:  0.04785687
## % pres/err predicted correctly:  -742.1662
## % of predictable range [ (model-null)/(1-null) ]:  0.02354854
## *****
## model index:  16
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##      3.1250      -0.2258
##
## Degrees of Freedom: 4136 Total (i.e. Null);  4135 Residual
## Null Deviance:      2735
## Residual Deviance: 2642  AIC: 2646
## log likelihood:  -1320.796
## Nagelkerke R2:  0.04601865
## % pres/err predicted correctly:  -742.3609
## % of predictable range [ (model-null)/(1-null) ]:  0.02329276
## *****
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
##      (Intercept)      stimlen      log_freq  stimlen:log_freq
##      4.14401      -0.24981      0.51886      -0.05161
##
## Degrees of Freedom: 4136 Total (i.e. Null);  4133 Residual
## Null Deviance:      2735
## Residual Deviance: 2643  AIC: 2651
## log likelihood:  -1321.628
## Nagelkerke R2:  0.0452055
## % pres/err predicted correctly:  -745.1049
## % of predictable range [ (model-null)/(1-null) ]:  0.01968732
## *****
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      log_freq

```

```
##      4.13374      -0.24527      0.09339
##
## Degrees of Freedom: 4136 Total (i.e. Null);  4134 Residual
## Null Deviance:      2735
## Residual Deviance: 2651 AIC: 2657
## log likelihood:  -1325.615
## Nagelkerke R2:   0.04130432
## % pres/err predicted correctly:  -746.2389
## % of predictable range [ (model-null)/(1-null) ]:  0.01819739
## *****
## model index:  15
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      4.3408      -0.2739
##
## Degrees of Freedom: 4136 Total (i.e. Null);  4135 Residual
## Null Deviance:      2735
## Residual Deviance: 2662 AIC: 2666
## log likelihood:  -1331.111
## Nagelkerke R2:   0.03591386
## % pres/err predicted correctly:  -747.9231
## % of predictable range [ (model-null)/(1-null) ]:  0.01598444
## *****
## model index:  14
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)
##      2.17
##
## Degrees of Freedom: 4136 Total (i.e. Null);  4136 Residual
## Null Deviance:      2735
## Residual Deviance: 2735 AIC: 2737
## log likelihood:  -1367.358
## Nagelkerke R2:   0
## % pres/err predicted correctly:  -760.0887
## % of predictable range [ (model-null)/(1-null) ]:  0
## *****
```

```
BestFLPModel<-FLPRes$ModelResult[[1]]
BestFLPModelFormula<-FLPRes$Model[[1]]

FLPAICSummary<-data.frame(Model=FLPRes$Model,
                          AIC=FLPRes$AIC,row.names=FLPRes$Model)
FLPAICSummary$DeltaAIC<-FLPAICSummary$AIC-FLPAICSummary$AIC[1]
FLPAICSummary$AICexp<-exp(-0.5*FLPAICSummary$DeltaAIC)
FLPAICSummary$AICwt<-FLPAICSummary$AICexp/sum(FLPAICSummary$AICexp)
FLPAICSummary$NagR2<-FLPRes$NagR2
```

```

FLPAICSummary <- merge(FLPAICSummary,FLPRes$CoefficientValues,
                        by='row.names',sort=FALSE)
FLPAICSummary <- subset(FLPAICSummary, select = -c(Row.names))

write.csv(FLPAICSummary,paste0(TablesDir,CurPat,"_",
                               RunLabel,"_",CurTask,
                               "_freq_len_pos_model_summary.csv"),row.names = FALSE)

kable(FLPAICSummary)

```

Model	AIC	DeltaAIC	AICw	NagR ²	Intercept	log_stimlen	log_freq	log_freq(I(pos^2))	log_freq(I(pos^2))	log_freq(I(pos^2))	log_freq(I(pos^2))	log_freq(I(pos^2))	log_freq(I(pos^2))	log_freq(I(pos^2))
preserved ~ stimlen * log_freq + I(pos^2) + pos	2603.002	0.000	0.000	0.527	0.169	0.474	0.463	0.505	0.294	-	NA	NA	0.028	0.769
					0.175	0.551	0.17	0.499	0.434	0.108				
preserved ~ stimlen * log_freq + (I(pos^2) + pos) *	2603.818	0.816	0.767	0.488	0.175	0.487	0.424	0.361	0.348	0.971	-	NA	0.084	0.763
					0.179	0.782	0.056	0.174	0.089	0.173			0.007	0.063
preserved ~ stimlen + I(pos^2) + pos + log_freq	2608.147	5.145	0.637	0.382	0.090	0.674	0.366	0.140	0.093	0.834	-	NA	0.030	0.506
					0.171	0.648	0.044	0.171	0.614					
preserved ~ stimlen * log_freq + pos	2608.222	5.220	0.320	0.102	0.137	0.183	0.063	0.259	0.519	0.577	-	NA	NA	NA
					0.156	0.608	0.051	0.160	0.976	0.542				
preserved ~ stimlen * log_freq + pos *	2609.601	6.599	0.238	0.196	0.325	0.891	0.414	0.356	0.526	0.385	-	NA	0.015	0.522
					0.161	0.118	0.061	0.120	0.159	0.661				
preserved ~ stimlen + (I(pos^2) + pos) *	2609.623	6.621	0.460	0.432	0.226	0.168	0.420	0.898	-	NA	-	0.091	0.007	0.026
					0.171	0.507	0.327	0.49	0.412	0.755			0.009	0.493
preserved ~ stimlen + pos + log_freq	2614.118	11.116	0.597	0.376	0.196	0.663	0.254	0.295	0.094	0.645	-	NA	NA	NA
					0.151	0.950	0.167	0.803						
preserved ~ stimlen * (I(pos^2) + pos)	2615.528	25.526	0.900	0.960	0.991	0.423	0.585	0.167	0.421	NA	0.958	0.529	NA	-
													0.142	0.779
														0.160
preserved ~ stimlen + pos *	2616.131	13.123	0.899	0.996	0.730	0.621	0.151	0.103	0.354	-	-	NA	NA	NA
								0.151	0.282	0.168	0.990	0.869		

Model	AIC Delta	AIC	AICw	NagR	Intercept	log_stimlen	log_freq	log_freq	I(pos^2)	pos	stimlen:log_freq	stimlen:I(pos^2)
preserved ~ stimlen + I(pos^2) + pos	2617.10	49.00	0.00	0.30	0.00	0.55	1.33	0.689	NA	NA	- NA	NA
					0.2006949		0.4494221					
preserved ~ stimlen * pos	2623.29	29.41	0.00	0.30	0.00	0.55	1.33	0.240	NA	NA	- NA	NA
						0.2726460	0.3837204					0.0246882
preserved ~ stimlen + pos	2623.20	30.40	0.00	0.30	0.00	0.55	1.33	0.237	NA	NA	- NA	NA
						0.1812481	0.1672667					NA
preserved ~ (I(pos^2) + pos) * log_freq	2626.26	25.93	0.00	0.30	0.00	0.55	1.33	0.225	NA	NA	- 0.0869064	0.0171839
							0.0225627	0.3818947		0.0098694		NA
preserved ~ pos + log_freq	2628.25	10.38	0.00	0.30	0.00	0.55	1.33	0.121	NA	NA	- NA	NA
							0.2130182					NA
preserved ~ pos * log_freq	2629.27	5.72	0.00	0.30	0.00	0.55	1.33	0.158	NA	NA	- NA	NA
							0.2170132	0.1572				NA
preserved ~ I(pos^2) + pos	2643.42	8.26	0.00	0.30	0.00	0.55	1.33	0.173	NA	NA	- NA	NA
							0.4183889					NA
preserved ~ pos	2645.42	3.90	0.00	0.30	0.00	0.55	1.33	0.112	NA	NA	- NA	NA
							0.2258403					NA
preserved ~ stimlen * log_freq	2651.45	2.54	0.00	0.30	0.00	0.55	1.33	0.518	NA	NA	- NA	NA
							0.2498079	0.0516147				NA
preserved ~ stimlen + log_freq	2657.23	2.73	0.00	0.30	0.00	0.55	1.33	0.093	NA	NA	- NA	NA
							0.2452650					NA
preserved ~ stimlen	2666.22	2.18	0.00	0.30	0.00	0.55	1.33	0.035	NA	NA	- NA	NA
							0.2738931					NA
preserved ~ 1	2736.73	1.70	0.00	0.30	0.00	0.55	1.33	0.008	NA	NA	- NA	NA

```
print(BestFLPModelFormula)
```

```
## [1] "preserved ~ stimlen * log_freq + I(pos^2) + pos"
```

```
print(BestFLPModel)
```

```
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)          stimlen          log_freq          I(pos^2)          pos  stimlen:log_freq
##      4.74746        -0.17555         0.50529         0.02880        -0.43411        -0.00000
##
## Degrees of Freedom: 4136 Total (i.e. Null);  4131 Residual
## Null Deviance:      2735
## Residual Deviance: 2591  AIC: 2603
```

```

# do a median split on frequency to plot hf/ls effects (analysis is continuous)
median_freq <- median(PosDat$log_freq)
PosDat$freq_bin[PosDat$log_freq >= median_freq] <- "hf"
PosDat$freq_bin[PosDat$log_freq < median_freq] <- "lf"

PosDat$FLPFitted <- fitted(BestFLPModel)

HFDat <- PosDat[PosDat$freq_bin == "hf",]
LFDat <- PosDat[PosDat$freq_bin == "lf",]

HF_Plot <- plot_len_pos_obs_predicted(HFDat, paste0(CurPat, " - ", RunLabel, " - High frequency"), "FLPFitted")

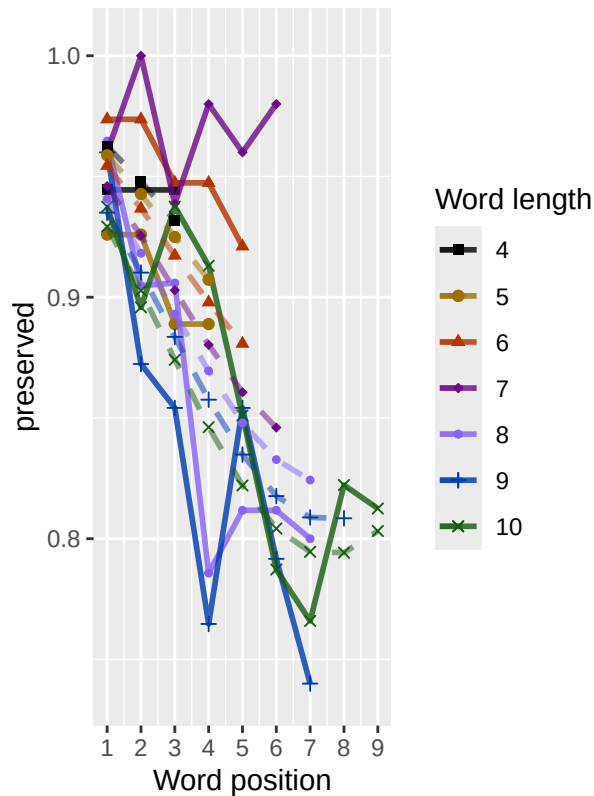
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.
LF_Plot <- plot_len_pos_obs_predicted(LFDat, paste0(CurPat, " - ", RunLabel, " - Low frequency"), "FLPFitted")

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.
library(ggpubr)
Both_Plots <- ggarrange(LF_Plot, HF_Plot) # labels=c("LF", "HF", ncol=2)

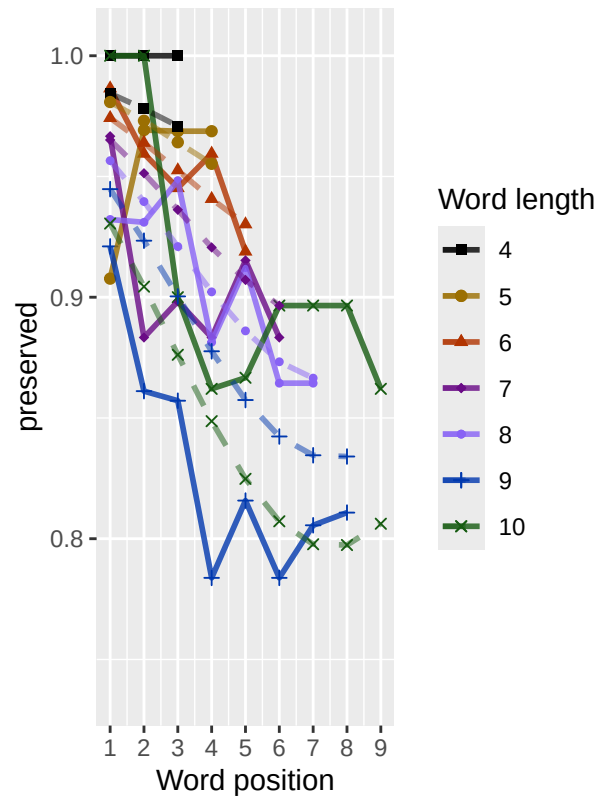
## Warning: Removed 1 row containing missing values or values outside the scale range (`geom_point()`).
## Warning: Removed 1 row containing missing values or values outside the scale range (`geom_line()`).
ggsave(paste0(FigDir, CurPat, "_", RunLabel, "_",
              CurTask, "_frequency_effect_length_pos_wfit.png"),
       device="png", unit="cm", width=30, height=11)
print(Both_Plots)

```

MS – no_frac – Low frequency



MS – no_frac – High frequency



```
# only main effects
MEModelEquations<-c(
  "preserved ~ CumPres",
  "preserved ~ CumErr",
  "preserved ~ (I(pos^2)+pos)",
  "preserved ~ pos",
  "preserved ~ stimlen",
  "preserved ~ 1"
)
MERes<-TestModels(MEModelEquations,PosDat)
```

```
## *****
## model index: 2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##        data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      3.014      -1.818
##
## Degrees of Freedom: 4136 Total (i.e. Null);  4135 Residual
## Null Deviance:      2735
## Residual Deviance: 1861 AIC: 1865
## log likelihood:  -930.3597
## Nagelkerke R2:  0.3937257
## % pres/err predicted correctly:  -487.2546
```

```

## % of predictable range [ (model-null)/(1-null) ]: 0.3584787
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) I(pos^2) pos
## 3.48470 0.02035 -0.41839
##
## Degrees of Freedom: 4136 Total (i.e. Null); 4134 Residual
## Null Deviance: 2735
## Residual Deviance: 2638 AIC: 2644
## log likelihood: -1318.914
## Nagelkerke R2: 0.04785687
## % pres/err predicted correctly: -742.1662
## % of predictable range [ (model-null)/(1-null) ]: 0.02354854
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) pos
## 3.1250 -0.2258
##
## Degrees of Freedom: 4136 Total (i.e. Null); 4135 Residual
## Null Deviance: 2735
## Residual Deviance: 2642 AIC: 2646
## log likelihood: -1320.796
## Nagelkerke R2: 0.04601865
## % pres/err predicted correctly: -742.3609
## % of predictable range [ (model-null)/(1-null) ]: 0.02329276
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen
## 4.3408 -0.2739
##
## Degrees of Freedom: 4136 Total (i.e. Null); 4135 Residual
## Null Deviance: 2735
## Residual Deviance: 2662 AIC: 2666
## log likelihood: -1331.111
## Nagelkerke R2: 0.03591386
## % pres/err predicted correctly: -747.9231
## % of predictable range [ (model-null)/(1-null) ]: 0.01598444
## *****
## model index: 1

```



```
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      1.7143      0.2055
##
## Degrees of Freedom: 4136 Total (i.e. Null);  4135 Residual
## Null Deviance:      2735
## Residual Deviance: 2678 AIC: 2682
## log likelihood: -1339.241
## Nagelkerke R2:  0.02791343
## % pres/err predicted correctly: -750.7998
## % of predictable range [ (model-null)/(1-null) ]:  0.01220479
## *****
## model index:  6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
##      2.17
##
## Degrees of Freedom: 4136 Total (i.e. Null);  4136 Residual
## Null Deviance:      2735
## Residual Deviance: 2735 AIC: 2737
## log likelihood: -1367.358
## Nagelkerke R2:  0
## % pres/err predicted correctly: -760.0887
## % of predictable range [ (model-null)/(1-null) ]:  0
## *****

BestMEModel<-MERes$ModelResult[[ BestModelIndexL1 ]]
BestMEModelFormula<-MERes$Model[[BestModelIndexL1]]

MEAICSummary<-data.frame(Model=MERes$Model,
                        AIC=MERes$AIC,row.names=MERes$Model)
MEAICSummary$DeltaAIC<-MEAICSummary$AIC-MEAICSummary$AIC[1]
MEAICSummary$AICexp<-exp(-0.5*MEAICSummary$DeltaAIC)
MEAICSummary$AICwt<-MEAICSummary$AICexp/sum(MEAICSummary$AICexp)
MEAICSummary$NagR2<-MERes$NagR2

MEAICSummary <- merge(MEAICSummary,MERes$CoefficientValues,
                    by='row.names',sort=FALSE)
MEAICSummary <- subset(MEAICSummary, select = -c(Row.names))

write.csv(MEAICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                        CurTask,"_main_effects_model_summary.csv"),
        row.names = FALSE)
kable(MEAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ CumErr	1864.7190	0.0000	1	1	0.3937253	0.014066	NA	-	NA	NA	NA
preserved ~ (I(pos^2) + pos)	2643.8297	79.1095	0	0	0.0478569	0.484701	NA	1.817862	0.0203483	-	NA
preserved ~ pos	2645.5937	80.8733	0	0	0.0460183	0.125046	NA	NA	NA	-	NA
preserved ~ stimlen	2666.2280	101.5018	0	0	0.0359139	0.340788	NA	NA	NA	NA	-
preserved ~ CumPres	2682.4817	17.7617	0	0	0.0279134	0.714313	0.2055355	NA	NA	NA	NA
preserved ~ 1	2736.7167	1.9971	0	0	0.0000000	0.169862	NA	NA	NA	NA	NA

```

if(DoSimulations){
  BestMEModelFormulaRnd <- BestMEModelFormula
  if(grepl("CumPres",BestMEModelFormulaRnd)){
    BestMEModelFormulaRnd <- gsub("CumPres","RndCumPres",BestMEModelFormulaRnd)
  }else if(grepl("CumErr",BestMEModelFormulaRnd)){
    BestMEModelFormulaRnd <- gsub("CumErr","RndCumErr",BestMEModelFormulaRnd)
  }

  RndModelAIC<-numeric(length=RandomSamples)
  for(rindex in seq(1,RandomSamples)){
    # Shuffle cumulative values
    PosDat$RndCumPres <- ShuffleWithinWord(PosDat,"CumPres")
    PosDat$RndCumErr <- ShuffleWithinWord(PosDat,"CumErr")
    BestModelRnd <- glm(as.formula(BestMEModelFormulaRnd),
                        family="binomial",data=PosDat)
    RndModelAIC[rindex] <- BestModelRnd$aic
  }
  ModelNames<-c(paste0("***",BestMEModelFormula),
                 rep(BestMEModelFormulaRnd,RandomSamples))
  AICValues <- c(BestMEModel$aic,RndModelAIC)
  BestMEModelRndDF <- data.frame(Name=ModelNames,AIC=AICValues)
  BestMEModelRndDF <- BestMEModelRndDF %>% arrange(AIC)
  BestMEModelRndDF <- rbind(BestMEModelRndDF,
                            data.frame(Name=c("Random average"),
                                         AIC=c(mean(RndModelAIC))))
  BestMEModelRndDF <- rbind(BestMEModelRndDF,
                            data.frame(Name=c("Random SD"),
                                         AIC=c(sd(RndModelAIC))))

  write.csv(BestMEModelRndDF,
            paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                  "_best_main_effects_model_with_random_cum_term.csv"),
            row.names = FALSE)
}

```

```

syll_component_summary <- PosDat %>%
  group_by(syll_component) %>% summarise(MeanPres = mean(preserved),
                                         N = n())
write.csv(syll_component_summary,paste0(TablesDir,CurPat,"_",
                                       RunLabel,"_",CurTask,

```

```

kable(syll_component_summary)                                     "_syllable_component_summary.csv"),row.names = FALSE)

```

syll_component	MeanPres	N
1	0.9180328	549
O	0.8913157	1923
P	0.9333333	30
S	0.8898305	236
V	0.8984989	1399

```

# main effects models for data without satellite positions

```

```

keep_components = c("0","V","1")
OV1Data <- PosDat[PosDat$syll_component %in% keep_components,]
OV1Data <- OV1Data %>% select(stim_number,
                             stimlen,stim,pos,
                             preserved,syll_component)
OV1Data$CumPres <- CalcCumPres(OV1Data)
OV1Data$CumErr <- CalcCumErrFromPreserved(OV1Data)

SimpSyllMEAICSummary <- EvaluateSubsetData(OV1Data,MEModelEquations)

```

```

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##          2.995      -1.892
##
## Degrees of Freedom: 3870 Total (i.e. Null); 3869 Residual
## Null Deviance:      2556
## Residual Deviance: 1754 AIC: 1758
## log likelihood: -876.9897
## Nagelkerke R2: 0.3870905
## % pres/err predicted correctly: -459.819
## % of predictable range [ (model-null)/(1-null) ]: 0.3518529
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)          pos
##          3.57856          0.02146         -0.44284
##
## Degrees of Freedom: 3870 Total (i.e. Null); 3868 Residual
## Null Deviance:      2556
## Residual Deviance: 2455 AIC: 2461

```

```

## log likelihood: -1227.506
## Nagelkerke R2: 0.05311852
## % pres/err predicted correctly: -691.4088
## % of predictable range [ (model-null)/(1-null) ]: 0.02611934
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) pos
## 3.1908 -0.2381
##
## Degrees of Freedom: 3870 Total (i.e. Null); 3869 Residual
## Null Deviance: 2556
## Residual Deviance: 2459 AIC: 2463
## log likelihood: -1229.466
## Nagelkerke R2: 0.05107492
## % pres/err predicted correctly: -691.5815
## % of predictable range [ (model-null)/(1-null) ]: 0.02587637
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen
## 4.4255 -0.2838
##
## Degrees of Freedom: 3870 Total (i.e. Null); 3869 Residual
## Null Deviance: 2556
## Residual Deviance: 2483 AIC: 2487
## log likelihood: -1241.668
## Nagelkerke R2: 0.03831223
## % pres/err predicted correctly: -697.8913
## % of predictable range [ (model-null)/(1-null) ]: 0.01700154
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) CumPres
## 1.7556 0.1994
##
## Degrees of Freedom: 3870 Total (i.e. Null); 3869 Residual
## Null Deviance: 2556
## Residual Deviance: 2510 AIC: 2514
## log likelihood: -1255.239
## Nagelkerke R2: 0.02402115
## % pres/err predicted correctly: -702.4394

```

```
## % of predictable range [ (model-null)/(1-null) ]: 0.01060465
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
## 2.172
##
## Degrees of Freedom: 3870 Total (i.e. Null); 3870 Residual
## Null Deviance: 2556
## Residual Deviance: 2556 AIC: 2558
## log likelihood: -1277.839
## Nagelkerke R2: 0
## % pres/err predicted correctly: -709.9791
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****
```

```
write.csv(SimpSyllMEAICSummary,
          paste0(TablesDir, CurPat, "_", RunLabel, "_",
                  CurTask,
                  "_OV1_main_effects_model_summary.csv"),
          row.names = FALSE)
kable(SimpSyllMEAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ CumErr	1757.9790	0.0000	1	1	0.3870902	3.994883	NA	- 1.891994	NA	NA	NA
preserved ~ (I(pos^2) + pos)	2461.0117	0.0317	0	0	0.0531183	5.578558	NA	NA	0.0214646	-	NA
preserved ~ pos	2462.9337	0.4953	0	0	0.0510749	1.190777	NA	NA	NA	-	NA
preserved ~ stimlen	2487.3357	29.3556	0	0	0.0383122	4.425519	NA	NA	NA	NA	-
preserved ~ CumPres	2514.4787	56.4987	0	0	0.0240211	1.755647	0.1994177	NA	NA	NA	NA
preserved ~ 1	2557.6779	99.6977	0	0	0.0000000	0.171936	NA	NA	NA	NA	NA

```
# main effects models for data without satellite or coda positions
# (tradeoff -- takes away more complex segments, in addition to satellites, but
# also reduces data)
```

```
keep_components = c("0", "V")
OVData <- PosDat[PosDat$syll_component %in% keep_components,]
OVData <- OVData %>% select(stim_number,
                           stimlen, stim, pos,
                           preserved, syll_component)
OVData$CumPres <- CalcCumPres(OVData)
OVData$CumErr <- CalcCumErrFromPreserved(OVData)

SimpSyllMEAICSummary2 <- EvaluateSubsetData(OVData, MEModelEquations)
```

```

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      2.877      -1.939
##
## Degrees of Freedom: 3321 Total (i.e. Null); 3320 Residual
## Null Deviance:      2241
## Residual Deviance: 1617 AIC: 1621
## log likelihood: -808.5517
## Nagelkerke R2: 0.3491084
## % pres/err predicted correctly: -429.6848
## % of predictable range [ (model-null)/(1-null) ]: 0.3140074
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)      pos
##      3.5597      0.0254      -0.4700
##
## Degrees of Freedom: 3321 Total (i.e. Null); 3319 Residual
## Null Deviance:      2241
## Residual Deviance: 2152 AIC: 2158
## log likelihood: -1076.013
## Nagelkerke R2: 0.05404044
## % pres/err predicted correctly: -610.2445
## % of predictable range [ (model-null)/(1-null) ]: 0.0264128
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      pos
##      3.1153      -0.2297
##
## Degrees of Freedom: 3321 Total (i.e. Null); 3320 Residual
## Null Deviance:      2241
## Residual Deviance: 2157 AIC: 2161
## log likelihood: -1078.627
## Nagelkerke R2: 0.05091651
## % pres/err predicted correctly: -610.6127
## % of predictable range [ (model-null)/(1-null) ]: 0.02582637
## *****
## model index: 5
##

```

```

## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      4.3382      -0.2783
##
## Degrees of Freedom: 3321 Total (i.e. Null); 3320 Residual
## Null Deviance:      2241
## Residual Deviance: 2179 AIC: 2183
## log likelihood: -1089.443
## Nagelkerke R2: 0.03793468
## % pres/err predicted correctly: -616.0513
## % of predictable range [ (model-null)/(1-null) ]: 0.01716388
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      1.8634      0.1503
##
## Degrees of Freedom: 3321 Total (i.e. Null); 3320 Residual
## Null Deviance:      2241
## Residual Deviance: 2224 AIC: 2228
## log likelihood: -1111.824
## Nagelkerke R2: 0.01080306
## % pres/err predicted correctly: -623.795
## % of predictable range [ (model-null)/(1-null) ]: 0.004829648
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
##      2.136
##
## Degrees of Freedom: 3321 Total (i.e. Null); 3321 Residual
## Null Deviance:      2241
## Residual Deviance: 2241 AIC: 2243
## log likelihood: -1120.652
## Nagelkerke R2: 0
## % pres/err predicted correctly: -626.8272
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****
write.csv(SimpSyllMEAICSummary2,
          paste0(TablesDir, CurPat, "_", RunLabel, "_", CurTask,
                 "_OV_main_effects_model_summary.csv"), row.names = FALSE)
kable(SimpSyllMEAICSummary2)

```

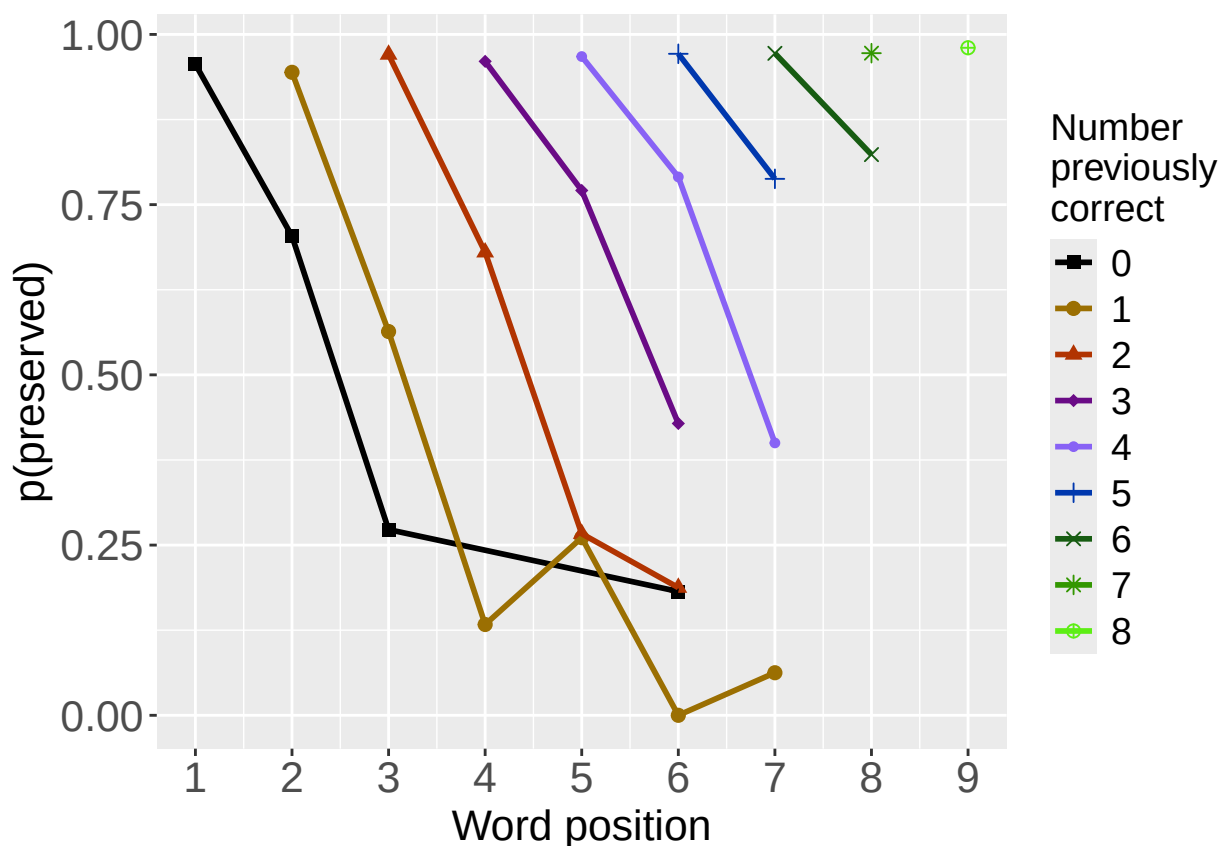
Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ CumErr	1621.1030	0.0000	1	1	0.3491082	2.877304	NA	-	NA	NA	NA
preserved ~ (I(pos^2) + pos)	2158.0265	36.9231	0	0	0.0540403	3.559719	NA	1.939196	0.0253951	-	NA
preserved ~ pos	2161.2534	40.1499	0	0	0.0509163	3.115264	NA	NA	NA	0.4700194	NA
preserved ~ stimlen	2182.8866	61.7826	0	0	0.0379347	4.338210	NA	NA	NA	0.2296853	-
preserved ~ CumPres	2227.6486	66.5443	0	0	0.0108031	8.63371	0.1502902	NA	NA	NA	0.2782912
preserved ~ 1	2243.3046	22.2007	0	0	0.0000000	0.135868	NA	NA	NA	NA	NA

```
# plot prev err and prev cor plots
```

```
PrevCorPlot<-PlotObsPreviousCorrect(PosDat,palette_values,shape_values)
```

```
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
```

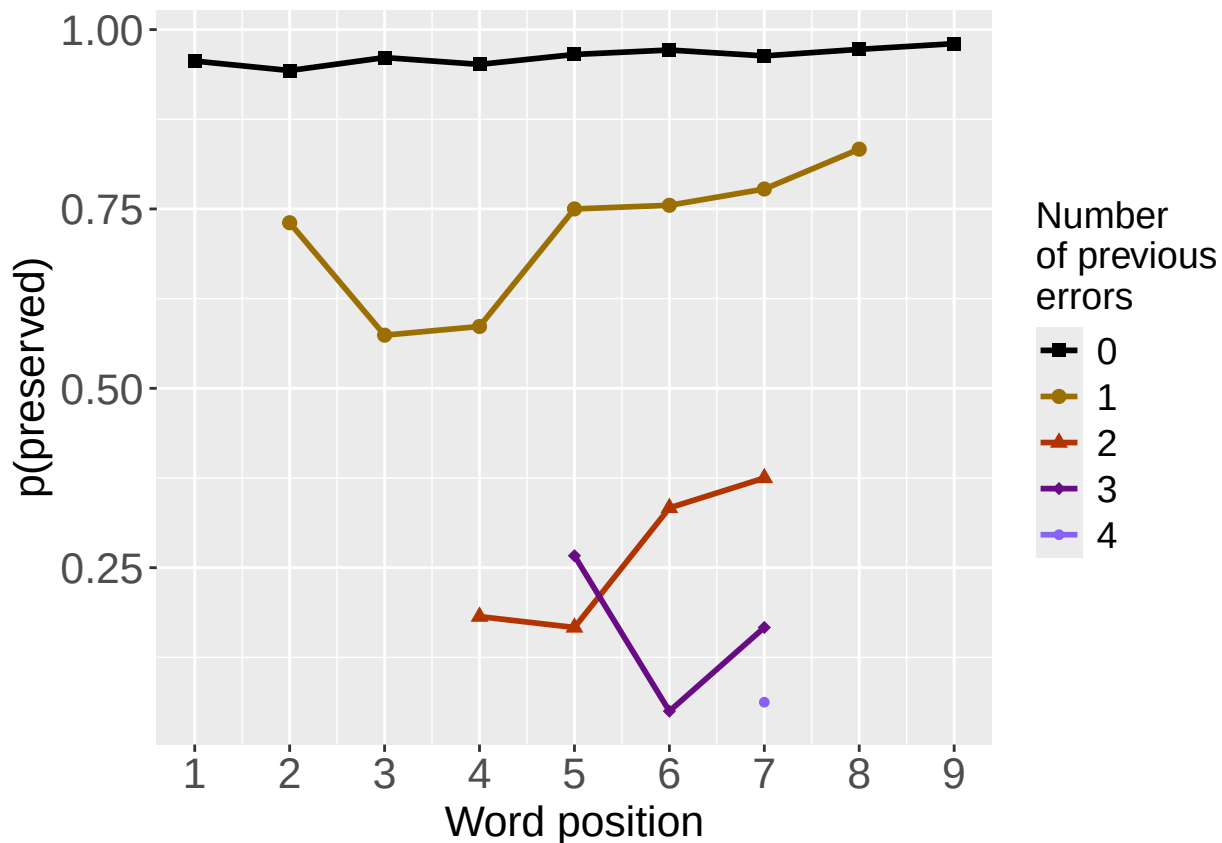
```
print(PrevCorPlot)
```



```
PrevErrPlot<-PlotObsPreviousError(PosDat,palette_values,shape_values)
```

```
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
```

```
print(PrevErrPlot)
```

```

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_PrevCorPlot_Obs")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevCorPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image

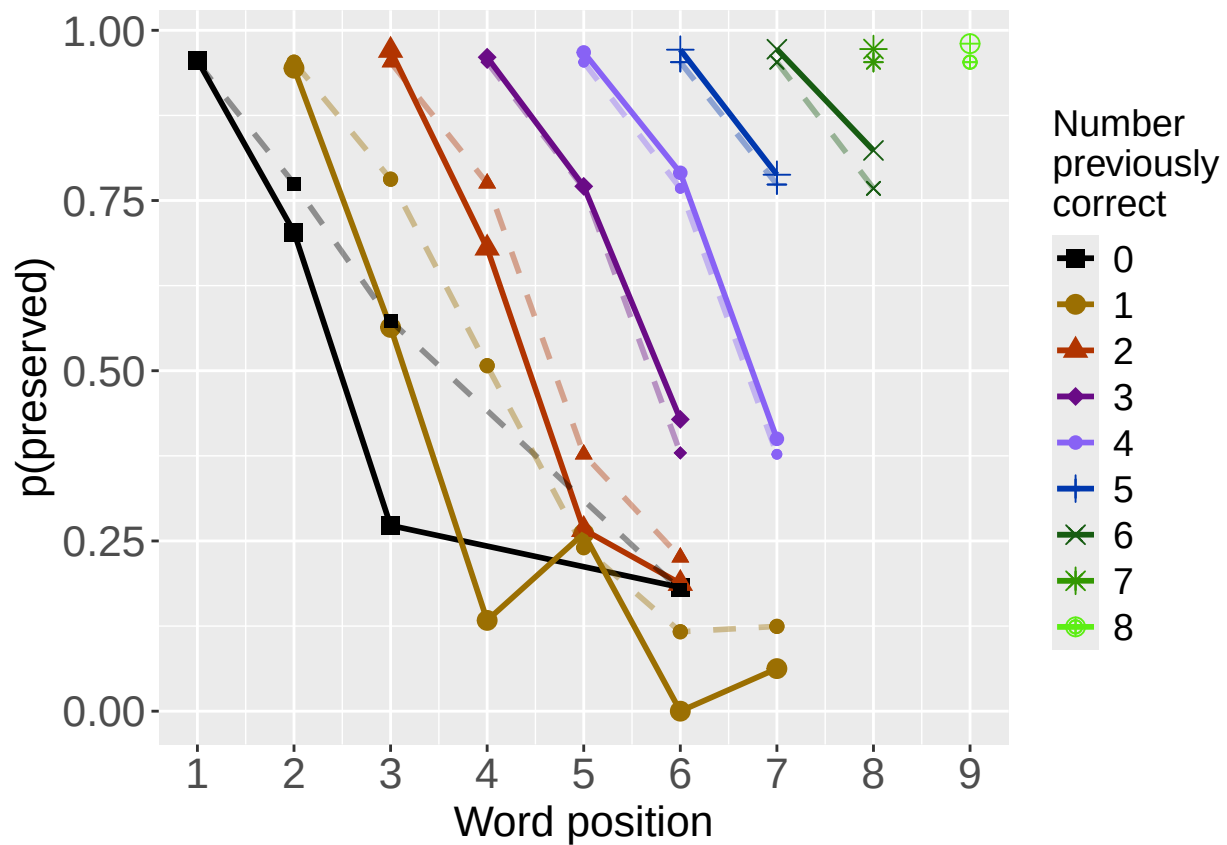
PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_PrevErrPlot_Obs")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevErrPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image
# plot prev err and prev cor with predicted values
MEModel<-MERes$ModelResult[[ BestModelIndexL1 ]]
PosDat$MEPred<-fitted(MEModel)

PrevCorPlot<-PlotObsPredPreviousCorrect(PosDat,"preserved","MEPred",palette_values,shape_values)

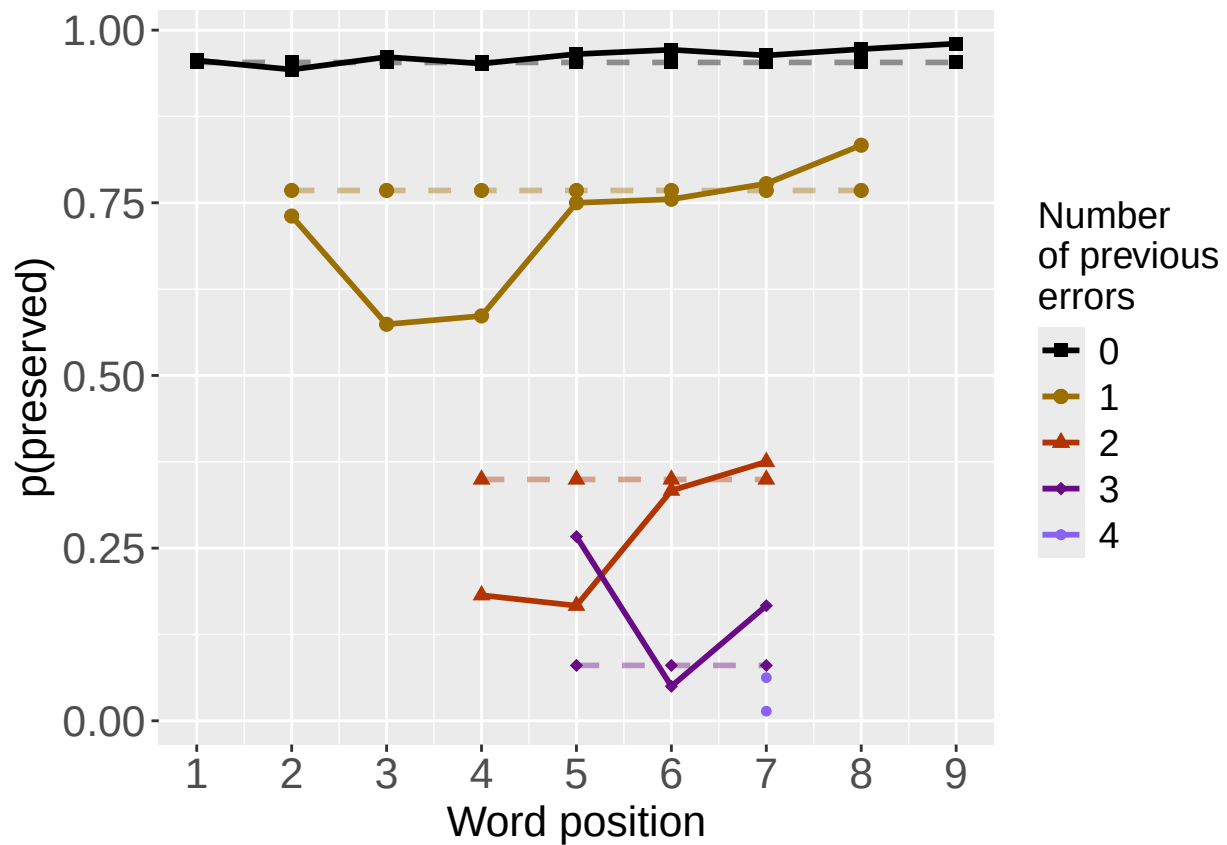
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
print(PrevCorPlot)

```



```
PrevErrPlot<-PlotObsPredPreviousError(PosDat,"preserved","MEPred",palette_values,shape_values)

## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
print(PrevErrPlot)
```



```

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",
                  CurTask,"PrevCorPlot_ObsPredME")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevCorPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",
                  CurTask,"PrevErrPlot_ObsPredME")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevErrPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image

```

```

CumAICSummary <- NULL

#####
# level 2 -- Add position squared (quadratic with position)
#####

# After establishing the primary variable, see about additions

BestMEFactor<-BestMEModelFormula
OtherFactor<-"I(pos^2) + pos"
OtherFactorName<-"quadratic_by_pos"

if(!grepl("pos",BestMEFactor,fixed = TRUE)){ # if best model does not include pos
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor,OtherFactorName)
  CumAICSummary <- AICSummary
  kable(AICSummary)
}

```

```

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      I(pos^2)         pos
##    3.27721    -1.94751     0.04667    -0.28511
##
## Degrees of Freedom: 4136 Total (i.e. Null);  4133 Residual
## Null Deviance:      2735
## Residual Deviance: 1839  AIC: 1847
## log likelihood:  -919.5921
## Nagelkerke R2:  0.4024157
## % pres/err predicted correctly:  -480.9718
## % of predictable range [ (model-null)/(1-null) ]:  0.3667337
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",

```

```

##      data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      3.014      -1.818
##
## Degrees of Freedom: 4136 Total (i.e. Null);  4135 Residual
## Null Deviance:      2735
## Residual Deviance: 1861  AIC: 1865
## log likelihood:  -930.3597
## Nagelkerke R2:  0.3937257
## % pres/err predicted correctly:  -487.2546
## % of predictable range [ (model-null)/(1-null) ]:  0.3584787
## *****
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)      pos
##      3.48470      0.02035      -0.41839
##
## Degrees of Freedom: 4136 Total (i.e. Null);  4134 Residual
## Null Deviance:      2735
## Residual Deviance: 2638  AIC: 2644
## log likelihood:  -1318.914
## Nagelkerke R2:  0.04785687
## % pres/err predicted correctly:  -742.1662
## % of predictable range [ (model-null)/(1-null) ]:  0.02354854
## *****

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	I(pos^2)	pos
preserved ~ CumErr + I(pos^2) + pos	1847.184	0.00000	1.0000000	0.9998443	0.4024157	3.277213	-1.947512	0.0466725	-0.2851118
preserved ~ CumErr	1864.719	17.53513	0.0001557	0.0001557	0.3937257	3.014066	-1.817862	NA	NA
preserved ~ I(pos^2) + pos	2643.829	796.64466	0.0000000	0.0000000	0.0478569	3.484701	NA	0.0203483	-0.4183889

```
#####
# level 2 -- Add length
#####

BestMEFactor<-BestMEModelFormula
OtherFactor<-"stimlen"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) CumErr stimlen
## 3.8378 -1.7756 -0.1072
##
## Degrees of Freedom: 4136 Total (i.e. Null); 4134 Residual
## Null Deviance: 2735
## Residual Deviance: 1854 AIC: 1860
## log likelihood: -926.7766
## Nagelkerke R2: 0.3966225
## % pres/err predicted correctly: -487.0539
## % of predictable range [ (model-null)/(1-null) ]: 0.3587424
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) CumErr
## 3.014 -1.818
##
## Degrees of Freedom: 4136 Total (i.e. Null); 4135 Residual
## Null Deviance: 2735
## Residual Deviance: 1861 AIC: 1865
## log likelihood: -930.3597
## Nagelkerke R2: 0.3937257
## % pres/err predicted correctly: -487.2546
## % of predictable range [ (model-null)/(1-null) ]: 0.3584787
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
```

```
## (Intercept)      stimlen
##      4.3408      -0.2739
##
## Degrees of Freedom: 4136 Total (i.e. Null);  4135 Residual
## Null Deviance:      2735
## Residual Deviance: 2662 AIC: 2666
## log likelihood:  -1331.111
## Nagelkerke R2:  0.03591386
## % pres/err predicted correctly:  -747.9231
## % of predictable range [ (model-null)/(1-null) ]:  0.01598444
## *****
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	stimlen
preserved ~ CumErr + stimlen	1859.553	0.00000	1.0000000	0.9297656	0.3966225	3.837818	-	-
preserved ~ CumErr	1864.719	5.16619	0.0755399	0.0702344	0.3937257	3.014066	-	NA
preserved ~ stimlen	2666.221	806.66798	0.0000000	0.0000000	0.0359139	4.340788	NA	-
								0.2738931

```
#####
# level 2 -- add cumulative preserved
#####

BestMEFactor<-BestMEModelFormula
OtherFactor<-"CumPres"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}
```

```
## *****
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      CumPres
##      2.6008      -1.8285      0.1812
##
## Degrees of Freedom: 4136 Total (i.e. Null);  4134 Residual
## Null Deviance:      2735
## Residual Deviance: 1833 AIC: 1839
## log likelihood:  -916.5324
## Nagelkerke R2:  0.4048768
## % pres/err predicted correctly:  -480.1021
## % of predictable range [ (model-null)/(1-null) ]:  0.3678764
## *****
## model index:  1
##
```

```
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      3.014      -1.818
##
## Degrees of Freedom: 4136 Total (i.e. Null);  4135 Residual
## Null Deviance:      2735
## Residual Deviance: 1861 AIC: 1865
## log likelihood: -930.3597
## Nagelkerke R2:  0.3937257
## % pres/err predicted correctly: -487.2546
## % of predictable range [ (model-null)/(1-null) ]:  0.3584787
## *****
## model index:  3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      1.7143      0.2055
##
## Degrees of Freedom: 4136 Total (i.e. Null);  4135 Residual
## Null Deviance:      2735
## Residual Deviance: 2678 AIC: 2682
## log likelihood: -1339.241
## Nagelkerke R2:  0.02791343
## % pres/err predicted correctly: -750.7998
## % of predictable range [ (model-null)/(1-null) ]:  0.01220479
## *****
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	CumPres
preserved ~ CumErr + CumPres	1839.065	0.00000	1.0e+00	0.9999973	0.4048768	2.600809	- 1.828479	0.1811950
preserved ~ CumErr	1864.719	25.65451	2.7e-06	0.0000027	0.3937257	3.014066	- 1.817862	NA
preserved ~ CumPres	2682.481	843.41622	0.0e+00	0.0000000	0.0279134	1.714313	NA	0.2055355

```
#####
# level 2 -- Add linear position (NOT quadratic)
#####

BestMEFactor<-BestMEModelFormula
OtherFactor<-"pos"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}
```



```

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      pos
##      2.632      -1.947      0.112
##
## Degrees of Freedom: 4136 Total (i.e. Null); 4134 Residual
## Null Deviance:      2735
## Residual Deviance: 1850 AIC: 1856
## log likelihood: -924.8846
## Nagelkerke R2: 0.3981501
## % pres/err predicted correctly: -484.3045
## % of predictable range [ (model-null)/(1-null) ]: 0.3623549
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      3.014      -1.818
##
## Degrees of Freedom: 4136 Total (i.e. Null); 4135 Residual
## Null Deviance:      2735
## Residual Deviance: 1861 AIC: 1865
## log likelihood: -930.3597
## Nagelkerke R2: 0.3937257
## % pres/err predicted correctly: -487.2546
## % of predictable range [ (model-null)/(1-null) ]: 0.3584787
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      pos
##      3.1250      -0.2258
##
## Degrees of Freedom: 4136 Total (i.e. Null); 4135 Residual
## Null Deviance:      2735
## Residual Deviance: 2642 AIC: 2646
## log likelihood: -1320.796
## Nagelkerke R2: 0.04601865
## % pres/err predicted correctly: -742.3609
## % of predictable range [ (model-null)/(1-null) ]: 0.02329276
## *****

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	pos
preserved ~ CumErr + pos	1855.769	0.000000	1.000000	0.9887394	0.3981501	2.631951	- 1.947281	0.1119644
preserved ~ CumErr	1864.719	8.950246	0.0113888	0.0112606	0.3937257	3.014066	- 1.817862	NA
preserved ~ pos	2645.593	789.823510	0.0000000	0.0000000	0.0460187	3.125046	NA	- 0.2258403

```
CumAICSummary <- CumAICSummary %>% arrange(AIC)
BestModelFormulaL2 <- CumAICSummary$Model[BestModelIndexL2]
write.csv(CumAICSummary, paste0(TablesDir, CurPat, "_",
                                RunLabel, "_", CurTask,
                                "_main_effects_plus_one_model_summary.csv"), row.names = FALSE)
kable(CumAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	I(pos^2)	pos	stimlen	CumPres
preserved ~ CumErr + CumPres	1839.066	0.000000	1.000000	0.9999973	0.4048765	2.600809	- 1.828479	NA	NA	NA	0.1811950
preserved ~ CumErr + I(pos^2) + pos	1847.180	0.000000	1.000000	0.9998443	0.4024153	2.77213	- 1.947512	0.0466725	-	NA	NA
preserved ~ CumErr + pos	1855.769	0.000000	1.000000	0.9887394	0.3981501	2.631951	- 1.947281	NA	0.1119644	NA	NA
preserved ~ CumErr + stimlen	1859.558	0.000000	1.000000	0.9297636	0.3966235	2.837818	- 1.775571	NA	NA	-	NA
preserved ~ CumErr	1864.719	7.535130	0.0001537	0.0001537	0.3937257	3.014066	- 1.817862	NA	NA	NA	NA
preserved ~ CumErr	1864.719	1.661900	0.0755399	0.0702341	0.3937257	3.014066	- 1.817862	NA	NA	NA	NA
preserved ~ CumErr	1864.719	5.654510	0.0000027	0.0000027	0.3937257	3.014066	- 1.817862	NA	NA	NA	NA
preserved ~ CumErr	1864.718	9.502460	0.0113888	0.0112606	0.3937257	3.014066	- 1.817862	NA	NA	NA	NA
preserved ~ I(pos^2) + pos	2643.827	996.644665	0.0000000	0.0000000	0.0478569	3.484701	NA	0.0203483	-	NA	NA
preserved ~ pos	2645.593	89.823510	0.0000000	0.0000000	0.0460187	3.125046	NA	NA	-	NA	NA
preserved ~ stimlen	2666.228	106.667983	0.0000000	0.0000000	0.0359130	3.340788	NA	NA	NA	-	NA
preserved ~ CumPres	2682.488	43.416200	0.0000000	0.0000000	0.0279134	3.714313	NA	NA	NA	NA	0.2055355

```
# explore influence of frequency and length

if(grepl("stimlen", BestModelFormulaL2)){ # if length is already in the formula
  Level3ModelEquations<-c(
    BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
    paste0(BestModelFormulaL2, " + log_freq")
  )
}
```

```

}else if(grepl("log_freq",BestModelFormulaL2)){ # if log_freq is already in the formula
  Level3ModelEquations<-c(
    BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
    paste0(BestModelFormulaL2," + stimlen")
  )
}else{
  Level3ModelEquations<-c(
    BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
    paste0(BestModelFormulaL2," + log_freq"),
    paste0(BestModelFormulaL2," + stimlen"),
    paste0(BestModelFormulaL2," + stimlen + log_freq")
  )
}
}

Level3Res<-TestModels(Level3ModelEquations,PosDat)

```

```

## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      CumPres      stimlen      log_freq
##      3.75473      -1.74681      0.23123      -0.16289      0.09799
##
## Degrees of Freedom: 4136 Total (i.e. Null); 4132 Residual
## Null Deviance:      2735
## Residual Deviance: 1804 AIC: 1814
## log likelihood: -901.9744
## Nagelkerke R2: 0.4165369
## % pres/err predicted correctly: -477.388
## % of predictable range [ (model-null)/(1-null) ]: 0.3714426
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      CumPres      stimlen
##      3.9730      -1.7558      0.2292      -0.1930
##
## Degrees of Freedom: 4136 Total (i.e. Null); 4133 Residual
## Null Deviance:      2735
## Residual Deviance: 1811 AIC: 1819
## log likelihood: -905.7258
## Nagelkerke R2: 0.4135401
## % pres/err predicted correctly: -477.4367
## % of predictable range [ (model-null)/(1-null) ]: 0.3713786
## *****
## model index: 3

```

```

##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      CumPres      log_freq
##      2.5909      -1.8005      0.1954      0.1331
##
## Degrees of Freedom: 4136 Total (i.e. Null);  4133 Residual
## Null Deviance:      2735
## Residual Deviance: 1818 AIC: 1826
## log likelihood: -909.075
## Nagelkerke R2:  0.41086
## % pres/err predicted correctly: -479.301
## % of predictable range [ (model-null)/(1-null) ]:  0.368929
## *****
## model index:  1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      CumPres
##      2.6008      -1.8285      0.1812
##
## Degrees of Freedom: 4136 Total (i.e. Null);  4134 Residual
## Null Deviance:      2735
## Residual Deviance: 1833 AIC: 1839
## log likelihood: -916.5324
## Nagelkerke R2:  0.4048768
## % pres/err predicted correctly: -480.1021
## % of predictable range [ (model-null)/(1-null) ]:  0.3678764
## *****
## model index:  2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
##      2.17
##
## Degrees of Freedom: 4136 Total (i.e. Null);  4136 Residual
## Null Deviance:      2735
## Residual Deviance: 2735 AIC: 2737
## log likelihood: -1367.358
## Nagelkerke R2:  0
## % pres/err predicted correctly: -760.0887
## % of predictable range [ (model-null)/(1-null) ]:  0
## *****

```

```

BestModelL3<-Level3Res$ModelResult[[ BestModelIndexL3 ]]
BestModelFormulaL3<-Level3Res$Model[[BestModelIndexL3]]

```

```

AICSummary<-data.frame(Model=Level3Res$Model,
                        AIC=Level3Res$AIC,
                        row.names = Level3Res$Model)
AICSummary$DeltaAIC<-AICSummary$AIC-AICSummary$AIC[1]
AICSummary$AICexp<-exp(-0.5*AICSummary$DeltaAIC)
AICSummary$AICwt<-AICSummary$AICexp/sum(AICSummary$AICexp)
AICSummary$NagR2<-Level3Res$NagR2

AICSummary <- merge(AICSummary,Level3Res$CoefficientValues,
                    by='row.names',sort=FALSE)
AICSummary <- subset(AICSummary, select = -c(Row.names))

write.csv(AICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                           CurTask,"_main_effects_plus_one_len_freq_summary.csv"),
          row.names = FALSE)

kable(AICSummary)

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	CumPres	log_freq	stimlen
preserved ~ CumErr + CumPres + stimlen + log_freq	1813.949	0.0000001	0.000000	0.9380099	0.4165363	1754732	-	0.2312308	0.0979901	-
							1.746810			0.1628851
preserved ~ CumErr + CumPres + stimlen	1819.453	5.502696	0.063841	0.059884	0.4135401	1972996	-	0.2292041	NA	-
							1.755794			0.1929705
preserved ~ CumErr + CumPres + log_freq	1826.150	2.201110	0.002241	0.002102	0.4108602	1590851	-	0.1954040	0.1331276	NA
							1.800539			
preserved ~ CumErr + CumPres	1839.063	5.116000	0.000003	0.000003	0.4048768	1600809	-	0.1811950	NA	NA
							1.828479			
preserved ~ 1	2736.710	22.76760	0.000000	0.000000	0.000000	0.169862	NA	NA	NA	NA

```

# BestModelIndex is set by the parameter file and is used to choose
# a model that is essentially equivalent to the lowest AIC model, but
# simpler (and with a slightly higher AIC value, so not in first position)
BestModelL3<-Level3Res$ModelResult[[ BestModelIndexL3 ]]
BestModelFormulaL3<-Level3Res$Model[BestModelIndexL3]

```

```

BestModel<-BestModelL3
BestModelDeletions<-arrange(dropterm(BestModel),desc(AIC))
# order by terms that increase AIC the most when they are the one dropped
BestModelDeletions

```

```

## Single term deletions
##
## Model:
## preserved ~ CumErr + CumPres + stimlen + log_freq
##      Df Deviance   AIC
## CumErr    1   2544.6 2552.6
## CumPres    1   1846.6 1854.6
## stimlen    1   1818.2 1826.2
## log_freq    1   1811.5 1819.5
## <none>      1804.0 1814.0

```

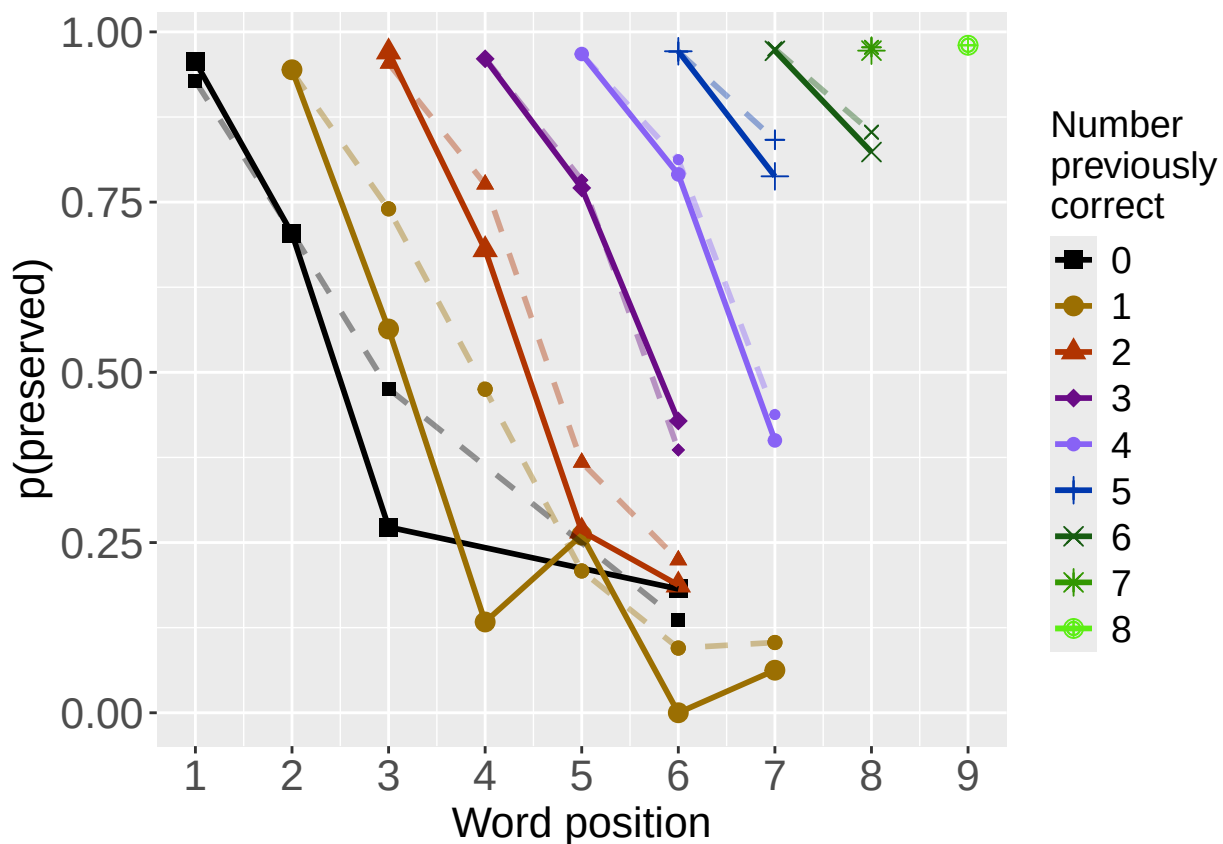
```
#####
# Single deletions from best model
#####

write.csv(BestModelDeletions,paste0(TablesDir,CurPat,"_",
                                   RunLabel,"_",CurTask,
                                   "_best_model_single_term_deletions.csv"),
          row.names = TRUE)

# plot prev err and prev cor with predicted values
PosDat$OAPred<-fitted(BestModel)

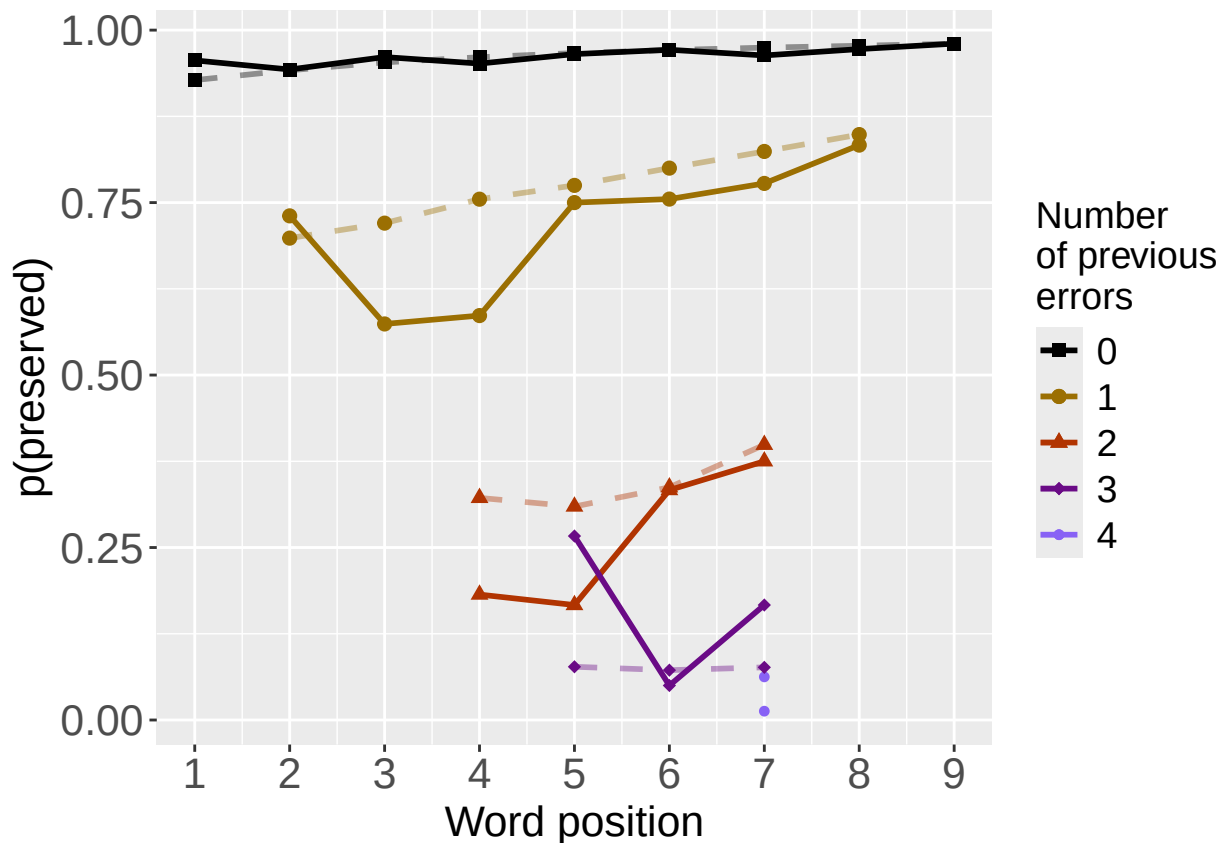
PrevCorPlot<-PlotObsPredPreviousCorrect(PosDat,"preserved","OAPred",palette_values,shape_values)

## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
print(PrevCorPlot)
```



```
PrevErrPlot<-PlotObsPredPreviousError(PosDat,"preserved","OAPred",palette_values,shape_values)

## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
print(PrevErrPlot)
```



```

PlotName<-paste0(FigDir,CurPat,"_",
  RunLabel,"_",CurTask,
  "_ObsPred_BestFullModel")
ggsave(filename=paste(PlotName,"_prev_correct.tif",sep=""),plot=PrevCorPlot,device="tiff",compression =

## Saving 6.5 x 4.5 in image
ggsave(filename=paste(PlotName,"_prev_error.tif",sep=""),plot=PrevErrPlot,device="tiff",compression = "

## Saving 6.5 x 4.5 in image
if(DoSimulations){
  BestModelFormulaL3Rnd <- BestModelFormulaL3
  if(grepl("CumPres",BestModelFormulaL3Rnd)){
    BestModelFormulaL3Rnd <- gsub("CumPres","RndCumPres",BestModelFormulaL3Rnd)
  }
  if(grepl("CumErr",BestModelFormulaL3Rnd)){
    BestModelFormulaL3Rnd <- gsub("CumErr","RndCumErr",BestModelFormulaL3Rnd)
  }

  RndModelAIC<-numeric(length=RandomSamples)
  for(rindex in seq(1,RandomSamples)){
    # Shuffle cumulative values
    PosDat$RndCumPres <- ShuffleWithinWord(PosDat,"CumPres")
    PosDat$RndCumErr <- ShuffleWithinWord(PosDat,"CumErr")
    BestModelRnd <- glm(as.formula(BestModelFormulaL3Rnd),
      family="binomial",data=PosDat)
    RndModelAIC[rindex] <- BestModelRnd$aic
  }
}

```

```

}
ModelNames<-c(paste0("***",BestModelFormulaL3),
              rep(BestModelFormulaL3Rnd,RandomSamples))
AICValues <- c(BestModelL3$aic,RndModelAIC)
BestModelRndDF <- data.frame(Name=ModelNames,AIC=AICValues)
BestModelRndDF <- BestModelRndDF %>% arrange(AIC)
BestModelRndDF <- rbind(BestModelRndDF,
                        data.frame(Name=c("Random average"),
                                   AIC=c(mean(RndModelAIC))))
BestModelRndDF <- rbind(BestModelRndDF,
                        data.frame(Name=c("Random SD"),
                                   AIC=c(sd(RndModelAIC))))
write.csv(BestModelRndDF,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                                "_best_model_with_random_cum_term.csv"),
          row.names = FALSE)
}

# plot influence factors
num_models <- nrow(BestModelDeletions) - 1
# minus 1 because last
# model is always <none> and we don't want to include that

if(num_models > 4){
  last_model_num <- 4
}else{
  last_model_num <- num_models
}

FinalModelSet<-ModelSetFromDeletions(BestModelFormulaL3,
                                     BestModelDeletions,
                                     last_model_num)

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_FactorPlots")

FactorPlot<-PlotModelSetWithFreq(PosDat,FinalModelSet,
                                 palette_values,FinalModelSet,PptID=CurPat)

## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##          3.014      -1.818
##
## Degrees of Freedom: 4136 Total (i.e. Null); 4135 Residual
## Null Deviance:      2735
## Residual Deviance: 1861 AIC: 1865
## log likelihood: -930.3597
## Nagelkerke R2: 0.3937257
## % pres/err predicted correctly: -487.2546
## % of predictable range [ (model-null)/(1-null) ]: 0.3584787

```



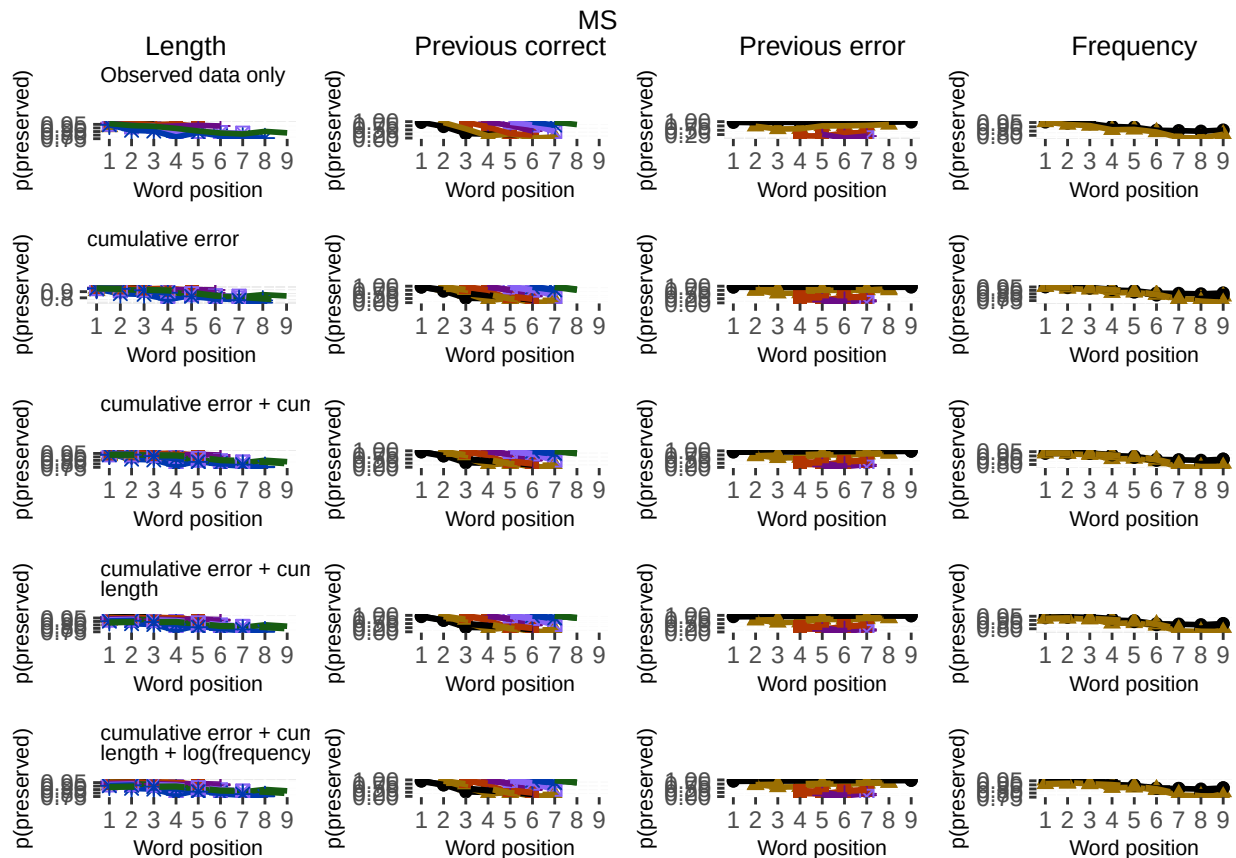
```

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      CumPres
##      2.6008      -1.8285      0.1812
##
## Degrees of Freedom: 4136 Total (i.e. Null); 4134 Residual
## Null Deviance:      2735
## Residual Deviance: 1833 AIC: 1839
## log likelihood: -916.5324
## Nagelkerke R2: 0.4048768
## % pres/err predicted correctly: -480.1021
## % of predictable range [ (model-null)/(1-null) ]: 0.3678764
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      CumPres      stimlen
##      3.9730      -1.7558      0.2292      -0.1930
##
## Degrees of Freedom: 4136 Total (i.e. Null); 4133 Residual
## Null Deviance:      2735
## Residual Deviance: 1811 AIC: 1819
## log likelihood: -905.7258
## Nagelkerke R2: 0.4135401
## % pres/err predicted correctly: -477.4367
## % of predictable range [ (model-null)/(1-null) ]: 0.3713786
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      CumPres      stimlen      log_freq
##      3.75473      -1.74681      0.23123      -0.16289      0.09799
##
## Degrees of Freedom: 4136 Total (i.e. Null); 4132 Residual
## Null Deviance:      2735
## Residual Deviance: 1804 AIC: 1814
## log likelihood: -901.9744
## Nagelkerke R2: 0.4165369
## % pres/err predicted correctly: -477.388
## % of predictable range [ (model-null)/(1-null) ]: 0.3714426
## *****
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.

```



```
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 4 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 4 rows containing missing values or values outside the scale range (`geom_point()`)
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes c
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`)
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes c
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 4 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 4 rows containing missing values or values outside the scale range (`geom_point()`)
ggsave(paste0(PlotName, ".tif"), plot=FactorPlot, width = 360, height=400, units="mm", device="tiff", compress=
# use \blandscape and \elandscape to make markdown plots landscape if needed
FactorPlot
```



```
DA.Result<-dominanceAnalysis(BestModel)
DAContributionAverage<-ConvertDominanceResultToTable(DA.Result)
write.csv(DAContributionAverage,paste0(TablesDir,CurPat,"_",RunLabel,"_",
CurTask,"_dominance_analysis_table.csv"),
row.names = TRUE)
kable(DAContributionAverage)
```

	CumErr	CumPres	stimlen	log_freq
McFadden	0.2962722	0.0205882	0.0171780	0.0063142

	CumErr	CumPres	stimlen	log_freq
SquaredCorrelation	0.1741575	0.0125250	0.0108691	0.0039211
Nagelkerke	0.1741575	0.0125250	0.0108691	0.0039211
Estrella	0.2100352	0.0143784	0.0116785	0.0043547

	deviance	deviance_explained
CumErr + CumPres + stimlen + log_freq	1803.949	930.7676
CumErr + CumPres + stimlen	1811.452	923.2649
CumErr + CumPres	1833.065	901.6516
CumErr	1860.719	873.9971
null	2734.716	0.0000

```
deviance_differences_df<-GetDevianceSet(FinalModelSet,PosDat)
write.csv(deviance_differences_df,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                                         CurTask,"_deviance_differences_table.csv"),
          row.names = FALSE)
deviance_differences_df
```

```
##
## CumErr + CumPres + stimlen + log_freq CumErr + CumPres + stimlen + log_freq 1803.949 930.7676 34.03525
## CumErr + CumPres + stimlen CumErr + CumPres + stimlen 1811.452 923.2649 33.76090
## CumErr + CumPres CumErr + CumPres 1833.065 901.6516 32.97057
## CumErr CumErr 1860.719 873.9971 31.95933
## null null 2734.716 0.0000 0.00000
##
## percent_of_explained_deviance increment_in_explained
## CumErr + CumPres + stimlen + log_freq 100.00000 0.8060761
## CumErr + CumPres + stimlen 99.19392 2.3220949
## CumErr + CumPres 96.87183 2.9711509
## CumErr 93.90068 93.9006781
## null NA 0.0000000
```

```
kable(deviance_differences_df[,2:3], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
kable(deviance_differences_df[,c(4:6)], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
NagContributions <- t(DAContributionAverage[3,])
NagTotal<-sum(NagContributions)
NagPercents <- NagContributions / NagTotal
NagResultTable <- data.frame(NagContributions = NagContributions, NagPercents = NagPercents)
names(NagResultTable)<-c("NagContributions","NagPercents")
write.csv(NagResultTable,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
```

	percent_explained	percent_of_explained_deviance	increment_in_explained
CumErr + CumPres + stimlen + log_freq	34.03525	100.00000	0.8060761
CumErr + CumPres + stimlen	33.76090	99.19392	2.3220949
CumErr + CumPres	32.97057	96.87183	2.9711509
CumErr	31.95933	93.90068	93.9006781
null	0.00000	NA	0.0000000

```

                                "_nagelkerke_contributions_table.csv"),row.names = TRUE)
NagPercents

```

```

##           Nagelkerke
## CumErr    0.86442246
## CumPres   0.06216712
## stimlen   0.05394822
## log_freq  0.01946221

```

```

sse_results_list<-compare_SS_accounted_for(FinalModelSet,"preserved ~ 1",PosDat,N_cutoff=10)

```

62

```

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumcor_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumcor_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumcor_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
```

63

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumcor_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
```

```
sse_table<-sse_results_table(sse_results_list)
write.csv(sse_table,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                          CurTask,"_sse_results_table.csv"),row.names = TRUE)
sse_table
```

	model	p_accounted_for	model_deviance	diff_CumErr	diff_CumErr+CumPres	diff_CumErr+CumPres+stimlen
## 1	preserved ~ CumErr	0.9573546	1860.719	0.000000000	-0.009420338	-0.01081135
## 2	preserved ~ CumErr+CumPres	0.9667750	1833.065	0.009420338	0.000000000	-0.00139102
## 3	preserved ~ CumErr+CumPres+stimlen	0.9681660	1811.452	0.010811358	0.001391020	0.00000000

model	p_accounted_for	model_deviance
preserved ~ CumErr	0.9573546	1860.719
preserved ~ CumErr+CumPres	0.9667750	1833.065
preserved ~ CumErr+CumPres+stimlen	0.9681660	1811.452
preserved ~ CumErr+CumPres+stimlen+log_freq	0.9683709	1803.949

model	diff_CumErr	diff_CumErr+CumPres	diff_CumErr+CumPres+stimlen	diff_CumErr+CumPres+stimlen+log_freq
preserved ~ CumErr	0.0000000	-0.0094203	-0.0108114	-0.0110163
preserved ~ CumErr+CumPres	0.0094203	0.0000000	-0.0013910	-0.0015959
preserved ~ CumErr+CumPres+stimlen	0.0108114	0.0013910	0.0000000	-0.0002049
preserved ~ CumErr+CumPres+stimlen+log_freq	0.0110163	0.0015959	0.0002049	0.0000000

```
## 4 preserved ~ CumErr+CumPres+stimlen+log_freq      0.9683709      1803.949 0.011016280      0.001595942      0.000204922
##   diff_CumErr+CumPres+stimlen+log_freq
## 1          -0.011016280
## 2          -0.001595942
## 3          -0.000204922
## 4           0.000000000
```

```
kable(sse_table[,1:3], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
write.csv(results_report_DF,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                                   CurTask,"_results_report_df.csv"),row.names = FALSE)
```

```
ncols_in_table <- ncol(sse_table)
kable(sse_table[,c(1,4:ncols_in_table)], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```